The MATL programming language

Luis Mendo

July 19, 2026

1 Introduction

MATL is a programming language based on MATLAB/Octave and suitable for code golf.

The MATL language is stack-oriented. Data are pushed onto and popped out of a stack. Functions may take elements from the stack (usually those at the top) and push one or more outputs onto the stack. Since the functions use data that's already present in the stack, reverse Polish notation (or postfix notation) is used.

This is in line with some other code-golf languages, and allows compact syntax for calling functions on data. To ease stack handling, values from the stack can also be copied and pasted using several clipboards. These are similar to variables in other stack-based code-golf programming languages.

The main goal in designing the language has been to keep it as close to MATLAB as possible. Think of it as MATLAB shorthand with data on a stack. MATL includes most commonly used MATLAB functions. It should be very easy for a MATLAB user to start programming in MATL within minutes.

1.1 The name

"MATL" /ˈmæt.ˌl/ is pronounced to rhyme with "cattle".

The spelling is purposely *short* (few letters) and *unusual* (words ending in "-tl" are rare in English). This is intended to reflect the facts that

- the language has been designed for code golf (*short* programs); and
- doing so requires some features that make it a somewhat weird/esoteric (*unusual*) language.

The name is written in capitals to follow MATLAB's official name, which is also all capitals. (Besides, that way it looks more important!)

1.2 Notation

This is how MATL code is displayed in this document: `iXyD`. Or sometimes with spaces for greater clarity: `i Xy D`.

MATLAB code is displayed like this: `n = input(''); disp(eye(n))`.

> Auxiliary or "meta" stuff is typeset in smaller font with indentation. This includes discussions as to why something has been done in a particular way, or things that remain to be done.

1.3 Examples

Here are some simple example programs, just to give an idea of MATL. These are explained in §12.

1. Infinite loop that does nothing:

   MATL: `‘T`

   MATLAB: `while true, end`

2. First 10 Fibonacci numbers:

   MATL: `1t8:"yy+`

   MATLAB:
   ```
   x=[0 1];for n=1:10,disp(x(2)),x=[x(2) sum(x)]; end
   ```

3. Get unique characters from an input string, maintaining their order:

   MATL: `ju`

   MATLAB: `unique(input('','s'),'stable')`

4. Same as above but done manually, that is, without using MATLAB's `unique` or its MATL equivalent `u`:

   MATL: `jt&=XRa~)`

   MATLAB:
   ```
   s=input('','s');s(~any(triu(bsxfun(@eq,s,s.'),1)));
   ```

2 The stack. Data types

The stack can be thought of as a vertical arrangement of elements. Those elements may be popped from the stack, pushed onto the top of the stack, or rearranged. Functions take stack elements as inputs, and push new elements as outputs.

Any element contained in the stack will be an *array* of one of the following types:

- Numerical arrays;
- Logical arrays;
- Character arrays;
- Cell arrays;
- Symbolic arrays.

Either of these arrays can be 2D or multidimensional. As in MATLAB, there aren't arrays of one dimension. All arrays are at least 2D, possibly with size 1 in some of their dimensions. Dimensions with size 1 are called singleton dimensions. Arrays with only one non-singleton dimension are referred to as *vectors*. For example, a row vector has size $1 \times n$. Numbers are considered 2D numerical arrays of size 1×1.

MATL does not have the `string` data type introduced by MATLAB's release 2016b. In this document, a *string* should be understood as a row vector of chars.

2D numerical arrays are full or sparse. Their default class is `double`. MATL can work with all the other numerical classes. Conversion is done with a function corresponding to MATLAB's `cast`, or with functions corresponding to `double` and `int64`. Symbolic arrays can be obtained converting from numerical arrays or strings, using a function corresponding to MATLAB's `sym` or `str2sym`.

> The symbolic data type was not originally in the language; it was a later addition, suggested by `@Sanchises`.

3 Statements and separators

A MATL program is divided into **statements**, **separators** and possibly **comments**.

A **statement** can be

- a literal;
- a function; or
- a control flow modifier (loops, conditional branches).

Literals can have a varying number of characters depending on their content. Functions and control flow modifiers consist of either

- one character (different from `X`, `Y` or `Z`); or
- two characters, the first of which is `X`, `Y` or `Z`.

Separators are sometimes needed to separate literals of the same type. For example, `12` means the number 12, whereas `1 2` means number 1 followed by number 2. Similarly, `'Padmé' 'Anakin'` means string `'Padmé'` followed by string `'Anakin'`, whereas `'Padmé''Anakin'` would be interpreted as a single string containing an apostrophe between the two names (quotation marks within strings are duplicated, as in MATLAB).

Separators are not needed if the literals are of different type, such as numerical and logical: for example: `T2` represents a MATLAB literal `T` (corresponding to `true` in MATLAB) followed by literal `2`.

Blank space and newline are the separator characters. For clarity (but not for code-golf purposes!), it may be convenient to display the program with spaces or linebreaks inserted between statements.

Using several lines also allows inserting **comments** in MATL code. The convention is similar to MATLAB: `%` marks the start of a comment, and then everything until the end of the current line is considered to be a comment (that is, ignored).

4 Literals

Literals can be

- Numbers (equivalent to 1×1 numerical arrays)
- Numerical arrays, 2D (numerical vectors and matrices)
- Logical arrays, 2D (logical vectors and logical matrices)
- Character arrays, 2D (strings or 2D character arrays)
- Cell arrays, 2D

The effect of a literal statement is to push the element represented by that literal onto the top of the stack.

Literals correspond to one of the data types referred to in §2 except symbolic, and are limited to 2D. It's not possible to define a multidimensional array directly as a literal (this limitation exists in MATLAB too). But it is possible, for example, to produce a 3D numerical array by concatenating several 2D arrays along the third dimension.

At any given time, the stack contents will consist of either literals or outputs produced by functions.

4.1 Numbers

Standard MATLAB formats are allowed, except that only `j` (not `i`) can be used for the imaginary unit, and only `e` (not `E`) can be used for exponents in scientific notation. Examples of valid number literals are `1.2`, `-.2j`, `1.`, `-1.2e3`, `+.25e-3j`.

Characters used in number literals "stick" to each other. Therefore consecutive number literals may need a separator to avoid confusion. For example, `12` represents number 12 (`1` sticks to `2`), whereas `1 2` represents numbers 1 and 2. An exception to numerical characters sticking is that characters `X`, `Y` or `Z` always grab the following character to form a two-character statement. For example `X123` represents function `X1` followed by number literal `23`.

The character `-` sticks to subsequent numbers. Thus `1-2` represents numbers 1 and -2 (`-` sticks to `2`), whereas `1- 2` represents number 1, function `-` and then number 2. On the other hand, `5.6e-7j24` is interpreted as number $5.6 \cdot 10^{-7} j$ followed by 24, even if no separator is included, because the first number literal is already complete when the `j` character is processed.

The character `+`, unlike `-`, does not stick to numbers. Thus `1+2` represents number 1, function `+` and then number 2.

`j` is interpreted as imaginary unit when used as part of literals, and as a function elsewhere. Again, a separator may be needed to avoid function `j` being interpreted as part of a literal. For example, `4j` is a literal representing number $4j$, whereas `4 j` is a literal representing number 4 followed by function `j`. To get number $j = \sqrt{-1}$ use the number literal `1j` (or the array `[j]`; see §4.2).

Similarly, `e` and `.` denote exponent or decimal point when used as part of literals, and are interpreted as functions elsewhere. For example, `4e3` is a literal representing number 4000, whereas `4e+` is a literal representing number 4, followed by function `e`, followed by function `+`.

Complex numbers in general can't be introduced directly as literals; only real or imaginary numbers can. For example, `1-2j` would be interpreted as literal `1` followed by literal `-2j`. (However, array literals allow infix operators, as discussed in §4.2; so `[1+2j]` would be interpreted as the complex number $1 + 2j$.)

4.2 Numerical arrays

Notation is the same as in MATLAB: `[1 2 3]`, `[1,2,3]`, `[1,2j;3e4,-.2e-5]`, `[]`. Within array literals commas can be used as column separators, in addition to spaces.

Colon notation can also be used: `[1:4;3 7 5 8]`, `[1:2:5;0 0 -1]`. For row vectors in colon notation the square brackets symbols may be omitted: `.5:.5:10`. In that case, a separator may be needed depending on what comes next.

The described infix colon notation can only be used in array literals. The colon symbol can also denote a function with similar meaning; but as happens for all

Table 1: Letters used as numbers in arrays

Letter	P	Y	N	O	l	H	I	K	A	B
Meaning	pi	inf	NaN	0	1	2	3	4	5	6

Letter	C	D	E	X	a	b	c	d	J	G
Meaning	7	8	9	10	-1	-2	-3	-4	1j	-1j

MATL functions, it takes its inputs from the stack, and thus uses postfix notation. A separator (space or newline) is sometimes needed to distinguish which use of the symbol `:` is meant. For example, `5:8` is a numerical array literal as described above, whereas `5: 8` (or `5 :8`) is the `:` function applied to number literal `5`, followed by number literal `8`.

`j` or `i` can be used to represent the imaginary unit within array literals (recall that only `j` can be used in number literals; §4.1). Thus `[0 j]`, `[0 1j]`, `[0 i]` and `[0 1i]` are equivalent.

Operators can be used within MATL literals as in MATLAB, and they are interpreted as infix. For example, the literal `[1/2 1+1/4]` would define an array with the two numbers 0.5 and 1.25.

Numerical arrays can also contain the following letters, which are interpreted as numbers:

- `P`, `Y`, `N`, which correspond to `pi`, `inf` and `NaN` respectively,
- `O`, `l`, `H`, `I`, `K`, `A`, `B`, `C`, `D`, `E`, `X`; `a`, `b` `c` `d`; `J`, `G`, respectively used as shorthand for numbers `0`, `1`, ..., `10`; `-1`, `-2`, `-3`, `-4`; `1j`, `-1j`.

Table 1 summarizes the correspondence between letters and numbers within literal arrays.

These letters need not have separators between them. Specifically, each letter is interpreted within a numerical array as the corresponding number *surrounded by spaces*. For example, `[1H2J2j;YNG42A]` corresponds to `[1 2 2 j 2j; inf NaN -j 42 5]`.

Only arithmetic operators, numbers, or the letters in Table 1 can appear within literals. They cannot contain calls to MATLAB functions (for example, `[cos(2); sqrt(3)]` is not allowed).

Similar restrictions apply to *evaluated input*, that is, array literals entered from keyboard by means of the `i` (`input`) function or by implicit input (see §7.3), or strings used as inputs to `U` (`str2num` / evaluate string), which are evaluated. However, in these cases the set of allowed letters is smaller: only `Y`, `N`, `P`, `T`, `F` are allowed; and *they need separators* (spaces are not automatically inserted).

Table 2 summarizes all characters that have *special meaning* within *number literals, array literals or evaluated input*. Note that in other uses (for example as

normal code) they have a different, possibly unrelated meaning.

Numerical array literals are full (as opposed to sparse). Although 2D sparse numerical arrays are supported, they cannot be introduced directly as literals.

> It's important to define carefully what is permitted within arrays. For instance: should these be allowed? `[sqrt(1:3)]`, `[cos(1) sin(1)]`, `[path]`. Allowing arbitrary MATLAB code within literals that can be evaluated brings flexibility, but it's dangerous. Consider `[rmpath('C:\path\to\files')]` for example: evaluating it would remove a folder from the path in MATLAB.
>
> One possibility to prevent that unwanted behaviour is to check the array contents strictly: a numerical array is `[...]` with contents separated by commas/spaces and semicolons. Each content must be a number, a numerical array, or a character array (which will be interpreted as numbers); and content sizes must match to form a 2D array. That involves hard work when parsing the array. It's much easier to let MATLAB do that work (via the compiled code), which it does very well.
>
> In fact, this problem already exists with MATLAB's `input` and `str2num` functions: they *evaluate* what the user types, which is a dangerous thing to do. If the user types `[cos(1) sin(2]` as input, evaluating that is probably fine. But what if they type `addpath('c:\path\to\folder')`?
>
> Possible solutions have been discussed in Stack Overflow[1]. The best approach seems to be: use regular expressions to detect function or variable names, but ignoring such names if they are within a string (for example, `['path' 115]` should be allowed, and would be equivalent to `'paths'`). The key here is that the regular expression doesn't need to check if the array is well formed (for example, `[[1 2; 3 4], [5 6]]` is not well formed), because that can be done later by evaluating it in MATLAB, once evaluation has been deemed safe. The regular expression only needs to make sure the array doesn't contain any reference to variables or functions, excluding string contents (a string can safely contain anything).
>
> The compiler proceeds as follows:
>
> - In array literals, the recognized letters that are not within strings are substituted by numbers (surrounded by spaces). An error is issued if either an unrecognized letter or two contiguous recognized letters are found that are not part of a string.
> - In evaluated input (user input or input to `U`), the recognized letters are considered as variables with the indicated value. An error occurs if an unrecognized letter or any two contiguous letters are found outside of a string.
>
> The procedure is different in the two cases because in the first case it is done at compile-time, whereas in the second it has to be done at run-time. So the check in the second case should be faster (it may have to be applied in a loop

[1] http://stackoverflow.com/q/33124078/2586922

Table 2: Characters that have special meaning within number literals or array literals

Character	Numerical literals	`[ ]` or `{ }` array literals	Evaluated input (`i`, implicit, `U`)
`T F`	—	"True", "false" †	"True", "false" ‡
`Y N P`	—	Infinity, not a number, pi †	Infinity, not a number, pi ‡
`0 1 H I K A B C D E X a b c d J G`	—	Numbers †	—
`j`	*Imaginary;* $\sqrt{-1}$	*Complex;* $\sqrt{-1}$	*Complex;* $\sqrt{-1}$
`i`	—	*Complex;* $\sqrt{-1}$	*Complex;* $\sqrt{-1}$
`e`	*Exponent*	*Exponent*	*Exponent*
`( )`	—	*Grouping*	*Grouping*
`& + < > ^ |`	—	*Operators (infix)*	*Operators (infix)*
`-`	*Sign*	*Operator (infix)*	*Operator (infix)*
`.`	*Decimal point*	*Decimal point; element-wise*	*Decimal point; element-wise*
`* / \`	—	*Matrix operations (infix)*	*Matrix operations (infix)*
`:`	—	*Colon operator*	*Colon operator*
`;`	—	*End row*	*End row*
`=`	—	*In relational operators*	*In relational operators*
`[ ]`	—	*Array concatenation*	*Array contatenation*
`{ }`	—	*Cell array concatenation*	*Cell array concatenation*
`~`	—	*In relational operators; "not"*	*In relational operators; "not"*

Italic text indicates that the meaning is the same as in MATLAB

† No separator needed

‡ Separator needed

for example). The need to keep the check simpler in the second case is the reason why fewer letters are allowed and separators are required.

The above also applies to checking the contents of cell arrays.

Note that MATLAB does not check the contents of `input` or `str2num` before evaluating them.

4.3 Logical arrays

`T` and `F` correspond to `true` and `false` respectively, and can be used for defining 2D arrays in MATLAB. So `[T F T; F F T]` or `[T F T;F F T]` define a logical 2D array. Column separators are not necessary, so the latter could be written as `[TFT;FFT]`.

For logical row vectors the square brackets can be omitted; that is, the notation `[T F T]` or `[TFT]` can be simplified to `TFT`. A separator may be needed if a new logical array follows: `TFT TT`. But it's not necessary in other cases: `TFT3.5`.

4.4 Character arrays

They are defined as in MATLAB. Quotation marks within strings need to be duplicated. Two example character arrays are `'I''m sorry, Dave'` (a row character array, or string) and `['Deckard'; 'Rachael']` (a 2D character array).

Numbers or number arrays can be used as components of character arrays; and then they are interpreted as ASCII codes of characters (as in MATLAB). For example, `['My food ' [105 115] ' problematic']` is the same as `'My food is problematic'`.

Strings (row character arrays) can also be defined using colon notation: for example, `'d':'j'` is equivalent to `'defghij'`, and `'a':4:'z'` is equivalent to `'aeimquy'`. As in MATLAB, the first and last operands must be of type char.

4.5 Cell arrays

They are defined as in MATLAB. Each cell can contain any of the preceding literals, or other cell arrays. An example cell array literal is `{'Great-Crack', 'But seeds are not pods!', {1,2,3,'travel','swift','petal'}}`.

For defining numbers within cell arrays, the same letter substitutions can be used as in numerical arrays (§4.2) or logical arrays (§4.3). For example, the array literal `{a:AX['hey';'ho!']TF}` is equivalent to `{-1:5 10 ['hey';'ho!'] true false}`

5 Functions

Functions operate on the contents of the stack, and produce new contents that are popped onto the stack. Function outputs can be of any of the types introduced in §2.

Functions take their input data in the same order in which they are in the stack, that is, bottom to top. So for example `5 4/` gives the result 1.25. This applies even if the input data are not taken from the top. Consider for example a two-input function. As will be seen in §5.2, this could take its inputs from arbitrary positions in the stack, say from the top and three positions below. In that case, the element that is *lowest* in the stack will be the *first* input to the function.

Outputs are pushed to the stack in the order in which they are returned by the function. That is, if the function returns two outputs, the first output will be pushed first; and then the second, which will be left on top.

There are four types of functions:

- **Normal functions** take their inputs from the stack and push their outputs onto the stack. An important particular class of normal functions is that of **indexing functions**, which will be dealt with separately.
- **Meta-functions** take their inputs from the stack, and produce no output on the stack. They are used for modifying the behaviour of normal functions; specifically, for defining their numbers of inputs and outputs.
- **Stack rearranging functions** take a group of elements from the stack and rearrange them; or get information about the stack.
- **Clipboard functions** give access to the clipboards, which store values from the stack and push them back. They behave like variables.

Inputs used by normal functions and meta-functions are always consumed, that is, disappear from the stack. For stack rearranging functions or clipboard functions that's not necessarily the case.

5.1 Normal functions

Normal functions operate on stack inputs and produce outputs that are pushed onto the stack. A given function may accept a variable number of inputs and produce a variable number of outputs.

The full list of normal functions is given in §8, and their detailed definitions appear in Appendix §A. Most MATL functions have a MATLAB equivalent. For example, MATL's `u` corresponds to MATLAB's `unique` (with the `'stable'` option by default). In MATL, each function has a *minimum and a maximum number of allowed inputs and outputs*, usually the same as in MATLAB; but in addition

default numbers of inputs and outputs are specified. Continuing with the example, `u` by default takes one input and produces one output. So if the top of the stack contains `[1 1 2 3 2]`, the function `u` would produce `[1 2 3]`. The meta-functions `$`, `#` and `&` (see §5.2) can be used to change the number of inputs and outputs of a function.

In some cases there exists a variation of the MATL function that corresponds to another one with a certain argument input fixed. For example, MATL's `Xu` corresponds to MATLAB's `unique(..., 'rows', ...)` (again, with the `'stable'` option by default).

MATL functions take inputs of the same data types as their corresponding MATLAB function. For example, in MATLAB numerical arrays can be added with character arrays or with logical arrays, and the result is a numerical array. Similarly, the MATL `+` function accepts these types of inputs and produces a numerical array as output. Thus `T1+D` would display a result 2 on screen (`D` by default displays the top of the stack), and `T'a'T++D` would display `99` (the character code for `'a'` is 97).

Some normal functions simply push fixed, predefined literals onto the stack depending on their input. These are functions `X0`,...,`Z9`. Also, clipboards initially contain certain predefined values. The former are used mainly for strings (such as `'stable'` or `'@mean'`), whereas the latter are used for numeric values (such as `2` or `[0 -1 1]`). These literals are used often, and calling the corresponding function requires less characters than actually typing the literal.

5.2 Meta-functions

Meta-functions are `$`, `#` and `&`. The first two are used for specifying the number of inputs and outputs that will be used by the next function, whether it's a normal, stack rearranging or clipboard function. The third specifies a certain number of inputs and outputs, as an alternative or secondary default input/output configuration for some functions.

The input/output specifications done with meta-functions `$`, `#` and `&` *are consumed (deleted)* by the next *normal, stack rearranging or clipboard function*. If that function doesn't allow the number of inputs or outputs specified by `$` or `#` an error occurs. Using `&` with a function that doesn't have a secondary default specification gives an error too. If a `$`, `#` or `&` specification is issued before the previous one has been consumed by a function, the new specification replaces the previous one.

The `$` meta-function **specifies the inputs** to be used by the next function. `$` takes as argument the top element of the stack, and consumes it. That element can be a *number* such as `3`, or a *logical array* such as `TTF`. In the first case, the next function will use the three highest elements on the stack as inputs. In the second case, the logical values are interpreted as a logical index into the stack elements,

starting from the top. So `TTF$` indicates that the function will use as inputs the two elements below the top element.

If the `$` specification is a logical array which is not a vector, it is interpreted in *column-major* order, that is, it's implicitly linearized. Also, *leading* `F` values are ignored. Thus, both `FTFT$` and `[F,F,F;F,T,T]$` would be equivalent to `TFT$`. This is in line with how MATLAB treats logical indices: they are automatically linearized and trailing `false` values are ignored. Note that here it's leading (not trailing) values that are ignored, because logical indexing into the stack is based on its top (not on its bottom): given $\cdots$`TF`, the rightmost value (`F`) refers to the top; the value to its left (`T`) refers to the second highest element in the stack, etc.

Consider as an example the `f` function, which corresponds to MATLAB's `find`. The code `[0 4 7 0]f` would produce the output `[2 3]`, just as `find([0 4 7 0)` in MATLAB. `[0 4 7 0]1 2$f` would produce 2, corresponding to the two-input MATLAB call `find([0 4 7 0], 1)`. The same result could be obtained with `[0 4 7 0]1TT$f`; and also with `[0 4 7 0]1FFFTT$f`.

For an example with non-contiguous inputs, assume the stack contains, bottom to top, `[0 4 7 0]`, `'abc'`, `1`. Then after executing `TFT$f` it will contain `'abc'`, `2`.

The `#` meta-function **specifies the outputs** to be produced by the following function. `#` takes as input either a number or a logical array. For ease of presentation, the case of logical arrays is discussed first. If the input is a logical array, it indicates which of the possible outputs will be asked for from that function. For example, if the stack contains `[0 4 7 0]`, `1` bottom to top, the code `2$FTT#f` will produce `4`, `2`, corresponding to `[~, col, val] = find([0 4 7 0], 1)` in MATLAB.

Again, logical arrays are interpreted in *column major order* when passed to `#`. However, in this case `F` values are *not ignored*. This is consistent with MATLAB's behaviour: `[ii,~] = find([0 4 7 0])` and `ii = find([0 4 7 0])` produce *different* results. In general, functions behave differently depending on the number of outputs with which they are called, regardless of whether some of those outputs will be ignored; and this is true even for ignored *trailing* outputs. Another example is the `size` function: `s = size(eye(3))` produces `[3 3]`, whereas `[s,~] = size(eye(3))` produces `3`. These would be done in MATL as `3XyZy` and `3XyTF#Zy` (`Xy` is `eye`, and `Zy` is `size`, which is called with one output argument by default).

> This may be a little surprising. One tends to think that adding outputs to a function call won't affect the preceding outputs. But in some cases it does. So saying "Use the first output of `find` to obtain the result" is not strictly correct, because it's ambiguous: "Which first output? The one I get when calling the function with two outputs? With one output? ...?".

If the input to `#` is a number n, there are two modes in which this number can be interpreted:

- *As number of inputs*: when n is in the range defined by the minimum and maximum numbers of outputs for the considered function, it indicates that the function will be called with n outputs. This is analogous to how `$` works with a number.

- *As a specific output*: when n exceeds the maximum allowed number of outputs M, it addresses the $(n-M)$-th output. It is equivalent to a logical vector with one `T` preceded by $n-M-1$ times `F`. As an example, for a function with a maximum number of outputs $M=3$, `5#` would be equivalent to `FT#`, and `6#` to `FFT#`.

  The "`6#`" notation in the preceding example is shorter, whereas "`FFT#`" is probably clearer.

Note that, even if the function inputs can be taken from arbitrary positions of the stack (using `$` with a logical array), the outputs are always pushed onto the top of the stack.

The `&` meta-function causes the next function to use its **alternative or secondary default input/output configuration**. Specifically, it sets an input specification and an output specification that are fixed for each function. For example, for `f` the meta-function `&` specifies 1 input and 2 outputs. So `[0 4 7 0]&f` would produce the arrays `[1 1]` and `[2 3]`.

When MATL processes a function, if no `$`, `#` or `&` specifications exist (for example, because they have been consumed by a previous function call), the default number of inputs or outputs is used for the current function. If `$`, `#` or `&` specifications are actually issued, they don't need to be right before the function. For example, there can be literals in between (but not another function, as it would consume those specifications). So, if the stack contains `[0 4 7 0]`, `1`, after executing `TTF$'abc'FTT#f` it will contain `'abc'`, `4`, `2`.

Input and output specifications can be modified by successive calls to meta-functions. For example, the code `1#2#f` is equivalent to `2#f` (the second call to `#` overrides the specification done by the first). The code `&2$f` causes `f` to use 2 inputs and 2 outputs: first `&` specifies 1 input and 2 outputs, and then `2$` specifies 2 inputs, leaving the number of outputs unaltered. On the other hand, `2$&f` is equivalent to `&f`, because `&` overrides the previous `2$`.

To delete the specifications set by `$`, `#` or `&`, and thus have the next function apply the default number of inputs or outputs, use `[]$` or `[]#`.

Another possibility would have been to define the language so that the number argument to `$` and `#` was interpreted as a bitmap indicating which inputs (and how many) are used. In other words, the number would be binary decoded and the result interpreted as a logical array. So `7$` in this alternative definition would correspond to `3$` or `TTT$` in the above definition: use as inputs the three highest elements, thus 111 in binary, which is number 7.

Pros of this alternative approach: most functions have three inputs/outputs or less, and this would thus result in more compact code (`TFT$` would become `5$`). Cons: (1) If a very large number of inputs were used the argument to `$` would become long. For example, MATLAB's `cat` often accepts a lot of inputs (maybe from a comma-separated list). 20 inputs would require number $2^{20}-1 = 1048575$ as argument to `$`. (2) Less natural semantics, specially for some stack rearranging functions.

So I think it's best to interpret integer numbers as number of inputs, and use logical arrays for bitmaps, as has been defined above. If desired, a number can be transformed into a logical vector corresponding to its binary expansion very easily (there's a function for that).

5.3 Stack rearranging functions

These functions rearrange the elements in the stack (by duplicating, moving, deleting) or get information about the stack (number of elements, stack contents). Except for `N` (see below), they don't strictly have outputs; rather, they operate on the stack elements directly, and any `#` specification other than the default `0#` will give an error.

- `x` (delete). It can have an arbitrary number $n = 0, 1, 2, \ldots$ or a logical array as `$` specification; default is 1. It deletes the n top elements from the stack; or the elements specified by the logical array. For example, `TTF$x` deletes the two elements below the top of the stack.

- `w` (swap). It can have an arbitrary number $n = 0, 1, 2, \ldots$ specified to `$`; by default 2. For $n > 0$ it swaps elements 1 (top) and n in the stack, leaving those in between intact. For $n = 0$ it does nothing. If the `$` specification is a logical array, the first and last indicated elements are swapped.

- `t` (duplicate elements). It can have an arbitrary number $n = 0, 1, 2, \ldots$ specified to `$`; by default 1. It copies the top n elements from the stack, and pastes them (without deleting them from their original positions) at the top of the stack, maintaining their order. For example, if the stack contains `'a'`, `'b'`, after `2$t` the stack will contain `'a'`, `'b'`, `'a'`, `'b'`.

  If a logical array is used as argument to `$`, it specifies which elements should be copied, maintaining their relative order. For example, `TTF$t` would copy the third- and second-highest elements and paste them at the top of the stack.

- `y` (duplicate element). This function is similar to `t`, but it only duplicates the lowest element among those specified as inputs. It can have an arbitrary number $n = 0, 1, 2, \ldots$ specified to `$`; by default 2; secondary default 3. Or a logical array can be used as input specification. For example, if the stack contains `'a'`, `'b'`, `'c'`, either `3$y` or `TTF$y` would leave the stack as `'a'`,

`'b'`, `'c'`, `'a'`. Without input specification, `y` duplicates the second element from the top.

- `b` (bubble up). It can have an arbitrary number $n = 0, 1, 2, \ldots$ specified to `$`; by default 3; secondary default 4. It makes the n-th highest element "bubble up" to the top, shifting elements $1, \ldots, n-1$ one position down. For $n = 0$ it does nothing.

  If a logical array is used as argument to `$`, the bubbling is done only among the indicated stack elements. For example, if the stack initially contains `'a'`, `'b'`, `'c'`, `'d'` from bottom to top, `TTFT$b` would leave the stack as `'b'`, `'d'`, `'c'`, `'a'`. Note how `'c'` remains in its original position.

- `N` (stack size). It pushes the number of elements that the stack currently has. It has no inputs (so the only allowed input specification would be `0$`, which is unnecessary anyway).

- `X#` (show stack). It displays the current contents of the stack (as a MATLAB cell array). This is useful for debugging.

5.4 Clipboard functions

Clipboards can be of two types: standard and automatic. Standard clipboards behave like variables: they allow copying values from the stack and pasting them (pushing them to the stack). Automatic clipboards are similar but copying is automatically triggered when certain events happen.

5.4.1 Standard clipboards

There are five standard clipboards (or arbitrarily many, depending on interpretation; more about that below). They are identified with the capital letters "H", ..., "L". Clipboards are used for copying from and pasting to the stack.

The first four clipboards behave as would be expected: they store values that can be retrieved later. Clipboard L is special: it consists of an arbitrarily large amount of "levels", and each level (beginning at 1) behaves as an independent clipboard. Thus clipboard L is actually an "infinite" collection of clipboards. Compared with clipboards H, ..., K, copying and pasting with clipboard L requires at least one extra character (to specify level), as will be seen.

Access to the clipboards is provided by means of the following functions:

- `XH`, `XI`, `XJ`, `XK` (copy to specified clipboard): copy a number of elements from the stack to one of the clipboards, without removing them from the stack. These functions have an arbitrary number of inputs $0, 1, 2 \ldots$, and no outputs. How many or which elements should be copied to the given clipboard is specified with `$`. For example, `2$XH` indicates copy the two

top elements to clipboard H; and `TF$XH` indicates copy the element right below the top. By default the top element is copied, corresponding to `1$`; secondary default (specified with `&`) is `2$`.

Copying elements to a clipboard removes its previous contents.

If the argument to `$` is 0, or a logical array containing only `F` values, the copying functions clear the clipboard contents.

- `H`, `I`, `J`, `K` (paste from specified clipboard): paste copied elements onto the stack, in the same relative order in which they originally were. The argument to `#` indicates which elements should be pasted, among those that the clipboard currently contains. By default all contents of the clipboard are pasted. These functions have no inputs.

- `XL` (copy to some level of clipboard L): behaves like `XH`, but takes one last additional input that specifies the level used within clipboard L. The default number of inputs is 2. So `1XL` (or `1 2$XL`, or `2$1XL`) copies the top of the stack to clipboard L, level 1. The input that specifies the level is automatically converted to `double`. So `TXL` would do the same thing.

  As usual, a logical array can also be used as argument `$`. If the stack contains from bottom to top `'a'`, `'b'`, `'c'`, the code `TTFT$4XL` copies `'a'`, `'b'` to clipboard L4 (note that the last `T` refers to `4`, which specifies the level).

- `L` (paste from some level of clipboard L): behaves like `H`, but takes as input one number that specifies the level used within clipboard L.

As can be seen from the above, clipboards H, I, J, K provide easier access (fewer characters) than clipboard L. In many cases four clipboards will be enough. On the other hand, clipboard L provides extended capabilities if needed, at the cost of at least one extra character to specify the level.

Clipboards H, I, J and K, as well as the first levels of clipboard L, initially contain the values indicated in Table 3.

> Using for example `H` instead of `2` may be useful to avoid a separator (space or newline) if that 2 is surrounded by other numbers. Values 0 and 1 can be obtained with functions `0` and `1` (which correspond to `zeros` and `ones` respectively; by default they take no inputs and produce a single number as output).

Clipboard L only has a certain number of levels initially (necessary to hold the initial contents specified in Table 3). Higher levels are created at run-time *on the fly*. For example, if clipboard L currently has 30 levels, copying some elements to level 32 will cause MATL to create levels 31 and 32: level 31 will be empty, and level 32 will hold the copied elements. Trying to paste from level 33 gives an error, because that level doesn't exist; but level 31 now exists, and is empty.

Therefore `31L` is valid code, and does nothing. `0#31L` is also valid, but `1#33L` (or even `F#33L`) is not, because a non-existent first element in level 33 is being referenced.

> Clipboard L effectively provides an infinite amount of "variables", but requires more characters compared with variables in other languages such as CJam. Clipboards H, I, J and K use less characters. Why only four such clipboards? The main reason is that MATLAB has a lot of "high-level" vectorized functions compared with other languages. It's desirable to have many of those functions in MATL, and that requires reserving a significant part of character space for them, at the expense of clipboards/variables. Also, characters should be reserved in order to be able to include new functions in the future. These four clipboards, plus the automatic ones, are probably enough in almost all situations. When that's not the case, clipboard L can be used with a small penalty in number of characters.
>
> Clipboard L starts at level 1, not 0, to match MATLAB indexing. This is important for debugging. The MATL debugger makes use of MATLAB's debugging capabilities to show relevant variables. Therefore the first level of clipboard L will be shown by MATLAB as 1, not as 0. A less important, more subtle advantage is that this makes the above mentioned implicit conversion to `double` unnecessary: characters are converted implicity by MATLAB, and `true` used as a logical index has the same effect as the integer index `1`; whereas `false` wouldn't have the desired effect of selecting the first (0-th) level.

5.4.2 Automatic clipboards

There are two automatic clipboards: G and M. These are multi-level clipboards, like L. A distinctive feature of clipboards G and M is that they don't have copying functions. Instead, copying happens automatically.

Clipboard G automatically stores all **user inputs**. User inputs may be obtained from explicit input functions (see §8) or implicitly (see §7.3). Each input is automatically stored on a different level of clipboard G, starting from 1. Clipboard G is initially empty, and each input fills a new level.

Function `G` is used for pasting from clipboard G. It can have 0 or 1 input arguments. With 0 input arguments, `G` retrieves the contents of all levels that currently exist in clipboard G. The default number of outputs is the current number of clipboard levels (each level contributes one output), although this can be of course modified by an output specification. The content of each level is pushed onto the stack separately. For example, if clipboard G currently has 2 levels, this produces 2 new elements in the stack.

With 1 input argument, `G` retrieves the contents of the clipboard level specified by the argument. The default number of outputs in this case is 1. For example,

Table 3: Initial content of clipboards

Clipboard	Contents
H	`2`
I	`3`
J	`1j`
K	`4`
L, level 1	`[1 2 1j]`
L, level 2	`[2 2 1j]`
L, level 3	`[1 1j-1]`
L, level 4	`[2 1j]`
L, level 5	`[1 0]`
L, level 6	`[2 1j-1]`
L, level 7	`[2 3 1]`
L, level 8	`[3 1 2]`
L, level 9	`[1 1j]`
L, level 10	`[1j -1 1]`
L, level 11	`3600`
L, level 12	`86400`
L, level 13	`1440`
L, level 14	`[31 28 31 30 ... 31]`
L, level 15	`[31 29 31 30 ... 31]`
L, level 16	Euler-Mascheroni constant, γ
L, level 17	Golden ratio, φ
L, level 18	`2j*pi`
L, level 19	`[1 3 2 4]`
L, level 20	`[1 3 4 2]`
L, level 21	`[3 1 2 4]`
L, level 22	`[3 1 4 2]`
L, level 23	`[3 4 1 2]`
L, level 24	`[1 .5j]`
L, level 25	`[1 .5+.5j]`
L, level 26	`[1+.5j 1j]`
L, level 27	`[.5+.5j 1j]`

`2G` retrieves the second user-input, assuming there have been two user-inputs. The selected user-input can also be specified using modular addressing, similar to indexing in arrays (see §5.5). So `0G` would retrieve the (currently) last user-input, `-1G` would retrieve the second-last input, etc.

If clipboard G has no levels (no user-input has occurred yet), calling `G` causes one user-input to be implicitly taken (see §7.3) in order to fill the first level.

This function by default has 0 input arguments if clipboard G currently has zero or one levels, and 1 input argument otherwise. This is intended to facilitate clipboard handling: when there are zero or one levels there is no need to specify which level to paste from. So without `$`, `#` or `&` specifications, function `G` behaves as follows: if no input has been asked from the user yet `G` takes one user-input implicitly and pushes it; if one user-input has been asked `G` pushes it; and if two or more user-inputs have been asked `G` takes one argument that specifies which one will be pushed. Secondary default (`&`) is 0 input arguments.

Clipboard M automatically stores all **function inputs** from the 4 most recent calls to *normal* functions that took *at least one* input from the stack. The inputs corresponding to each function call are stored on a different level, by increasing order of age. So clipboard level 1 contains all inputs of the most recent call (to a normal function that used inputs); level 2 from the second most recent call, etc.

Function `M` is used for pasting from clipboard M. It takes one numeric input, which can be interpreted in two different ways depending on its value:

- *Combined addressing*: `M` takes as input a number 1, ..., 4. The number defines the addressed level. The default number of outputs is the number of elements stored in that level (that is, the number of inputs of the function call used). So for example `[3 7 0 5]u1Mf` produces the vectors `[3 7 0 5]` (`unique([3 7 0 5])`) and `[1 2 4]` (`find([3 7 0 5])`); and `3 4+ 1M*` produces `7` and `12`.

  If the referred function call hasn't actually occurred, `M` produces no outputs. For example, `3M` does nothing if there have been less than three normal function calls with inputs. However, any output specification other than `0` gives rise to an error in this case.

- *Individual addressing*: `M` takes as input a number 5, 6, ... The number individually addresses inputs from function calls that used more than one input. Addressing is from most recent to less recent call, and from most recent (rightmost) to less recent (leftmost) input in the function signature. Only calls that had more than one input are considered, because single-input calls are already covered by combined addressing. So 5 refers to the latest input of the latest call that used several inputs; 6 refers to the second latest such input, etc.

  If the addressed function input doesn't actually exist, `M` produces no output.

For example, `5M` does nothing if there haven't been any normal function calls with several inputs.

As an example, consider the following code, where spaces are used to separate statements for clarity: `10 20 30 3$+ [5 0 7] 1 4 X0 3$f =`. There are four function calls, with the following inputs and outputs:

1. `3$+` (addition): inputs are `10`, `20`, `30`. Produces `60`.
2. `X0` (predefined literal): input is `4`. Produces the string `'last'`.
3. `3$f` (`find`): inputs are `[5 0 7]`, `1`, `'last'`. Produces `3`.
4. `=`: inputs are `60`, `3`. Produces `F` (`false`).

In this scenario, `4M` would push `10`, `20`, `30` onto the stack. `5M` would push `3`. `9M` would push `[5 0 7]`. `10M` would push `30`, no `4`, because `4` was a single input (which could be produced with `3M`).

Note the similarity of `M` with function `L`, which is used for pasting from multi-level clipboard L. In both cases, the input argument to the function specifies the clipboard level which is being addressed, and the output specification controls which of the multiple elements (arrays) existing in that level are pushed to the stack. In function `G`, on the other hand, both the input argument and the output specification specify which levels are pushed, each level containing only one element (array). Another difference between clipboards G and M is that the former uses chronological order, whereas the latter uses reverse chronological order.

Clipboards G and M differ from H, ..., L in that here copying is done automatically, and it doesn't use or consume any `$` or `#` specifications. But pasting with functions `G` and `M` does consume `$` and `#` specifications, like pasting from standard clipboards does.

> The default number of inputs and outputs of `G` has been defined such that, in normal usage, an input number is used to address the intended level; but this number is spared when it's not needed because there is only one level. Note that with this function there may be several ways of obtaining the same result using different input and output specifications. For example, if clipboard G has two levels, `0$1#G` is equivalent to `1$1G`.
>
> Perhaps a cleaner syntax could have been used to distinguish combined addressing and individual addressing in clipboard M, such as negative numbers for individual addressing, two inputs, or a different function name. However, all those alternatives either require using more characters in the code or consume more character space.

5.5 Indices and indexing functions

Indexing with **round brackets** is used in MATLAB for accessing array's contents and cells of a cell array. Indexing with **curly braces** is used for accessing cells' contents in a cell array.

Indexing can be used for two purposes: **referencing** elements of an array, or **assigning** values to elements of an array. For example, in the statement `a(2,:) = b(1,:)`, reference indexing is being used for `b`, and assignment indexing is being used for `a`.

The four basic indexing functions in MATL are the following:

- `)` round-bracket reference indexing;
- `(` round-bracket assignment indexing;
- `X)` curly-brace reference indexing;
- `X(` curly-brace assignment indexing.

> I find it more natural to use a closing parenthesis `)` for reference indexing and an opening parenthesis `(` for assignment indexing. The latter suggests "opening" the array so it can accept new contents.

In addition, there are four functions that include a colon index. This is useful for indexing rows or columns of 2D arrays:

- `Y)` round-bracket reference indexing with final colon;
- `Z)` round-bracket reference indexing with initial colon;
- `Y(` round-bracket assignment indexing with final colon;
- `Z(` round-bracket assignment indexing with initial colon;

There are also two functions that correspond to MATLAB's reference indexing with a colon only:

- `X:` round-bracket reference indexing with colon;
- `Y:` curly-brace reference indexing with colon.

The arrays used as indices, in turn, may be of one of the following four types. The are all analogous to different forms of MATLAB indexing, with some extensions.

- Numerical indexing;
- Logical indexing;

- `end`-based indexing;
- Colon indexing;

With **numerical** indexing, each number specifies the coordinates of the element in a given dimension (or in several coalesced dimensions if linear indexing is used; see below). Indexing is 1-based, as in MATLAB. If the values are not integer, they are automatically *rounded* to the closest integer. In addition, they are interpreted *modularly* or *semi-modularly*, as is now described:

- *For reference indexing*, values can be outside the range corresponding to the size of the array, and are interpreted as *modular indices*, via a 1-based modulo operation. For example, given a dimension of size 5, either -1 or 9 used as indices are equivalent to 4 (adding or subtracting 5 an integer number of times gives 4, which is in the range $1,\dots,5$).
- *For assignment indexing*, only *non-positive* values are interpreted modularly. This will be called *semi-modular indexing*. Positive values exceeding the size of the array are interpreted normally (not modularly), and will effectively extend the array. For example, given a dimension of size 5, index -1 is equivalent to 4. Index 9 is interpreted as 9, so assigning content to that position will make the array grow.

With **logical** indexing, the index is a logical array in which `T` (`true`) indicates elements that should be picked in the corresponding dimension (or dimensions). For example, the integer index `[1 3 4]` would be equivalent to the logical index `TFTT`. In MATLAB, a logical index need not have the same size as the dimension(s) it's indexing into. `a(2,[true false true true])` works even if `size(a,2)` is greater than 4; and trailing `false` values are ignored[2]. This also applies to MATL.

For `end`**-based indexing** the following convention is used: whenever an array used as index has *three or less* elements and at least one of them has *non-zero imaginary part* it is interpreted as an `end`-based index. A complex value such as `b*j+a` corresponds to `b*end + a`, and if the array has two or three elements colons are implicitly assumed between them. So, for example,

- `[2 j-1]` represents `2:end-1`.
- `.5j` represents `end/2`.
- `[1 2 j]` represents `1:2:end`.

An `end`-based index is equivalent to a numerical index, with `end` expanded depending on the dimensions of the indexed array. The rounding is applied to the

[2] http://stackoverflow.com/a/2462898/2586922

value that results from interpreting the `end` part (imaginary) and adding with the normal part (real). For example, the index `.5j+.8` applied on an array with 7 elements would give the fourth element: `.5j` is first translated into `3.5`, then `.8` is added to give `4.3`, and this number is rounded to `4`. Similarly, the modular or semi-modular conversion (for reference or assignment indexing respectively) is applied at the end.

Colon indexing refers to `:` used as an index. Often, a colon index is the same as the `end`-based index `1:end`, that is, `[1 1j]`. However, there are cases when a colon is specifically needed (such as in null assignment, to be described later). Although MATL doesn't have an equivalent of `:` to be specified as index, the *string* `':'` can be used instead of a colon, as in MATLAB[3].

All these four types of indices can be used with any of the indexing functions, taking into account that numeric values are interpreted as modular (in reference indexing) or semi-modular (in assignment indexing).

> Common `end`-based indexing expressions are predefined as initial contents of clipboard L; see Table 3.
>
> I had initially dropped the idea to include `end`-based indexing because it seemed complicated: MATLAB's `end` is used *within* an array, and the known size of that array determines its meaning. But MATL's stack-oriented character and absence of infix notation means that the `end`-based array needs to be pushed onto the stack by itself, and exist independently of whichever array it will end up indexing into.
>
> But I picked it again following a discussion with `@beaker`. I came up with the idea that the imaginary unit can be used as an "`end` label", which is left unresolved until an indexing function such as `)` establishes its relationship with a specific array. By using imaginary values the sign is available to indicate positive or negative displacements with respect to `end`.
>
> Modular indexing was added later, following a suggestion by `@Dennis`. At first I thought it would not be compatible with `end`-based indexing, but it could be made compatible by restricting modular indices to be scalar.
>
> Then I changed the semantics of `end`-based indices so that any numeric index could be interpreted modularly, and also added rounding.
>
> Assignment indexing was initially not modular at all. Then I added semi-modularity (modularity for non-positive values). Assignment indexing cannot be fully modular because then it would not be possible to grow an array by assigning a new entry.
>
> Element-wise indexing (see §5.5.1) was added later.

Quite independently of the above, an index can refer to one or several dimensions. This gives rise to **multidimensional**, **linear** or **partially linear** indexing, exactly like in MATLAB. "Linearity" of an index refers to the fact that several dimensions are coalesced and indexed jointly, using the standard column-major

[3] `http://stackoverflow.com/q/22537326/2586922`

order. In general, if there are less indices than dimensions, the last index refers to all trailing dimensions as if they were collapsed into one.

5.5.1 Basic indexing functions

`)` takes an arbitrary number of inputs $2, 3, 4, \ldots$, by default 2, and produces 1 output by default. The first input is the array to be indexed, and the others are interpreted as indices into that array. For example, `a(2,[1 3 4])` would be done in MATL, assuming `a` is currently at the top of the stack, as `2[1 3 4]3$)`. The three inputs would be consumed, and the indexed array `a(2,[1 3 4])` would now occupy the top of the stack.

As an example with modular indexing, if the top of the stack is a 6×7 array `a`, `8[3 -1]3$)` would produce `a(2,[3 6])`.

The function `)` can also be used with 2 outputs. The simplest case is when only one input index is used. This is the secondary default input/output configuration. The first output is the same as described above, and the second is the "complementary" array formed by the index values not present in the input index. For example, `[2 4 6 8][2 3]2#)` produces the arrays `[4 6]` and `[2 8]`, as does `[2 4 6 8]FTT2#)`.

In general, the second output of `)` is obtained by means of a *null assignment* of the form `a(...) = []`, which removes the indicated elements from the input array. In MATLAB, given a 3×4 array `a`, neither `a(1:3,[2 4]) = []` nor even `a(1:end,[2 4]) = []` are allowed. Instead, a *colon* needs to be explicitly used: `a(:,[2 4]) = []`. The general rule is that, if several indices are used, all but one need to be `:` (colon). As mentioned earlier, the string `':'` can be used in MATL (as in MATLAB) as equivalent to a colon. Thus, `':'[2 4]3$2#)` is valid code, which produces as first output an array with columns 2 and 4 from the input, and as second output the input array with those columns removed.

`(` takes an arbitrary number of inputs, by default 3, secondary default 4. The first input is the array to which values will be assigned; the second is an array containing the values to be assigned to the first; and the following inputs are the indices into the first array. Consider for example `a(5,6) = b`. In MATL, if the stack contains `a` and `b` on top, the corresponding assignment would be `5 6 4$(`. Or, combining this with reference indexing into the right-hand size, `a(5,[6 7]) = b(8,[9 10])` would be `8 [9 10] 3$) 5 [6 7] 4$(`. Note that the initial `8 [9 10] 3$(` replaces the top of the stack `b` by `b(8, [9 10])`.

The order of inputs used by `(`, namely "destination", "data", "indices", naturally adapts to certain situations, such as processing in a loop: in each iteration some processing is done, resulting in the data to be assigned to a destination array that was previously pushed onto the stack. The index of the assignment is then pushed onto the stack and `(` is called.

The function `(` can also do null assignments, corresponding to MATLAB's `a(...) = []`, to remove elements from an array. The simplest case is a row or

column array (numeric, character or cell) with a single index. For example, `{10 20 30}[][1 3](` produces the cell array `{20}`. In the general case, null assignments to whole rows or columns of a 2D array (or to slices of a multidimensional array) have the same restrictions mentioned above: all indices but one must be `':'`. Thus, to remove the second row of a 2D array the following could be used: `[]2':'4$(`. However, in most cases this can be done more easily with the functions `Y(` and `Z(`, to be described below.

Of course, the inputs to `(` and `)` can be specified by logical arrays (`TTT$`) instead of numbers (`3$`).

`X)` is analogous to `)`, but corresponds to MATLAB's `{...}` reference indexing. It takes an arbitrary number of inputs $2, 3, 4, \ldots$, by default 2. The first input should be a cell array, and the others are interpreted as indices into that cell array. The indices can be integer or logical values. The result of that indexing in MATLAB would be a comma-separated list of the indexed cell's contents. In MATL, each element of that list is pushed onto the stack in the indexing order.

`X(` is similar to `(`, but corresponds to MATLAB's `{...}` assignment indexing; and has some differences. In MATLAB, an assignment like `a{[1,2]} = ...` can be done in two ways:

- `[a{[1,2]}] = b{[2001,2010]}` (comma-separated list generated from a cell array on the right-hand side);
- `[a{[1,2]}] = deal('Follow the white rabbit', '101')` (`deal` on the right-hand side).

The first is actually equivalent to `(...)` indexing: it can be done as `a(...) = b(...)` or, more generally, `a(...) = reshape(b(...),...)`. The second makes more sense for MATL, and is actually what `X(` does.

Considering two indices for greater generality, a typical assignment of this type might be, in MATLAB, `[a{[1,2],[3,4]}] = deal({'Mostly harmless', [],42,5})`. In MATL it's done as follows. `X(` takes as inputs, in this order: a destination cell array (corresponding to `a`), an arbitrary number of data (4 in this case), an arbitrary number of indices (2 in this case) that define the exact destination of those data within the cell array, and *a number that specifies how many indices exist*. So in this case 8 inputs should be specified for `X(`. If the stack contains, bottom to top: destination cell array, `'Mostly harmless'`, `[]`, `42`, `5`, `[1 2]`, `[3 4]`, `2`, then `8$X(` does the assignment.

The reason why the number of indices needs to be specified as an additional input is to avoid ambiguities with `end`-based indexing. Suppose the number of indices were not specified, and consider the following inputs to `X(`: destination cell array, `'Mostly harmless'`, `[]`, `42`, `5`, `[1 j]` (note that the latter is an index interpreted as `1:end`). The destination cell array (`a`) initially has size $2 \times 1 \times 2$. In these conditions, there are *two* possible interpretations for the assignment: either `[a{1:end}] = deal({'Mostly harmless',[],42,5})` (as-

sign four values, using linear indexing) or `[a{42,5,1:end}] = deal({'Mostly harmless',[]})` (assign two values along the third dimension).

> This ambiguity occurs only because of `end`-based indexing. With standard numeric or logical indices, interpreting one input as index instead of data would increase or maintain the number of indexed positions in the destination array, while at the same time would reduce the amount of available data, so there would only be (at most) one valid possibility.
>
> Having to specify the number of indices is not very nice, specially compared with `(`, where it's not needed. But it seems unavoidable. And after all, MATLAB also requires `deal` in this case, which is not needed in round-bracket assignments.

According to the above, the minimum number of inputs for `X(` would be 4: destination array, at least one data element, at least one index, and number of indices. The most common case is probably that of a single data element and a single index. To alleviate the syntax in this case, this can be realized with only 3 inputs, omitting the last one specifying the number of indices. That is, when `X(` is used with 3 inputs they are interpreted as destination array, one data element and one index. This is the default number of inputs for `X(`. Any other number above 3 is also allowed, and in that case the last input tells the number of indices.

> What a mess! Couldn't this `X(` stuff have been made simpler?
>
> I would have liked to, but I haven't found a way to make curly-brace assignment simpler while keeping consistence with `$` semantics. MATLAB notation `[a{ind1,ind2}] = deal(data1, data2, data3, ...)` is not terribly simple to begin with; and having everything in a stack can sometimes obscure things a bit (although of course it has other advantages).
>
> In many cases the assignment will be to a single cell with a single index (`a{ind} = data`). In MATL that case is simple indeed, and totally analogous to round-bracket indexing: push `a`, then `val`, then `ind`, and `X(` will do the assignment. In the more general case one only has to remember to include the number of indices as a final input.
>
> All in all, notation may get a little complicated in the general case. On the positive side, some people may consider code obfuscation to be, well, a good thing... specially in a weird/esoteric language worth that name!
>
> On a more serious note, MATL indexing supports integer/logical indexing; multidimensional/linear/partially linear indexing; curly braces (cell contents) and round brackets (cells, standard arrays); reference (getting values from an array) and assignment (putting values into an array); all working on a stack. I hope notation is just about as complicated as it needed to be.

Functions `)`, `(`, `X)` and `X(` also support an *element-wise indexing* mode: with a single indexing input, if that input is a cell array of numeric arrays, those arrays are interpreted as indices into the first input, but the indices are first combined to

form a *linear index*. The behaviour is similar to that of MATLAB's `sub2ind`, but with modularity (for `)` and `X)`) or semi-modularity (for `(` and `X(`) and rounding. Modularity and rounding are applied per dimension (that is, before converting to a linear index).

For example, `[10 20 30; 40 50 60][1 2][3 -1]3$)` gives `[30 20; 60 50]` (that is, all pairs of entries from `[1 2]` and `[3 -1]` are used as indices; the latter is modularly equivalent to `[3 2]`); whereas `[10 20 30; 40 50 60]{[1 2] [3 -1]})` gives `[30 50]` (matching pairs of entries from `[1 2]` and `[3 -1]` are used as indices). The latter is equivalent to `[10 20 30; 40 50 60][5 4])`; effectively, the indices `[1 2]` and `[3 -1]` are combined into the linear index `[5 4]` (taking into account the size of `[10 20 30; 40 50 60]`; modularly and with rounding per dimension). The same result would be achieved with `[10 20 30; 40 50 60]{[1 2] [3;-1] 3.3})`, as `3.3` in the third dimension is in this case modularly equivalent to `1`. This also illustrates that non-scalar arrays within the indexing cell array are reshaped to match the first (so in this case `[3;-1]` is reshaped into `[3 -1]`), and that scalar indices are implicitly repeated (so `3.3` is the same as `[3.3 3.3]`. Scalar data values are also repeated if needed, that is, they are written in all specified positions. Similarly, in `X(` if there is a single data value or array it is repeated if necessary.

When using this mode with `X(`, the number of indices (possibly specified by the last input of `X(`) is `1`, which refers to the cell array, even if that cell array contains several numeric arrays.

As an example with assignment indexing, `[10 20 30; 40 50 60][7 8]{[-2 3] [4 -1]}(` gives `[10 20 30 0; 40 50 60 7; 0 0 8 0]`. First the initial 2×3 matrix is extended into a 2×4 matrix to include a `7` at row 2 (semi-modularly equivalent to -1), column 4. This is then extended into a 3×4 matrix to include an `8` at row 3, column 3 (semi-modularly equivalent to -1, considering the *updated* matrix size resulting from the previous step).

5.5.2 Colon indexing functions

`Y)` is similar to `)`, except that a colon index is added *after* the specified index or indices. It takes an arbitrary number of inputs; by default 2, corresponding to the array and one specified index. By default it produces 1 output. For example, if the stack contains the array `[1 2; 3 4]`, the code `1Y)` produces the first row of the array, that is, `[1 2]`. So it's equivalent to the more cumbersome `1':'3$)` or `1[1 j]3$)` (the latter uses `[1 j]`, or `1:end`, instead of `:`, but they *are* equivalent for reference indexing).

`Z)` is analogous to `Y)` except that the colon index is added *before* the specified index or indices. Thus if the stack contains the array `[1 2; 3 4]`, the code `1Z)` produces the first column of the array, `[1;3]`. The equivalent code using `)` would be `':'1 3$)` or `[1 j]1 3$)`.

The functions `Y)` and `Z)` also have 2-output versions, analogous to that of `)`. In this case only one input index can be different from `':'`, and the second output is the "complementary" array. For example, `[1 2; 3 4; 5 6] [1 3]2#Y)` produces the arrays `[1 2; 5 6]` and `[3 4]`. The secondary default configuration for these functions is 1 input and 2 outputs.

`Y(` and `Z(` add a final or initial colon respectively in assignment indexing. They take at least 2 inputs; by default 3, corresponding to destination array, data, and one specified index. They produce 1 output. For example, if the stack contains `[1 2; 3 4]`, the code `5,1Y(` produces the array `[5 5; 3 4]`. It is thus equivalent to the more cumbersome `5 1':'4$(` or `5 1[1 j]4$(`.

The functions `Y(` and `Z(` support null assignment, with the usual restriction that only one input index can be different from `':'`. Thus, given an array `[1 2; 3 4]`, the code `[]1Z(` removes its first column, producing `[2; 4]`. The corresponding code with `(` would be `[]':'1 4$(` (for null assignment `:` and `1:end` are *not* equivalent).

`X:` takes an input array and applies MATLAB's `(:)` reference indexing. This has the effect of linearizing the array into a column. The number of inputs and outputs is 1. For example, `[1 2; 3 4]X:` would replace the input array `[1 2; 3 4]` by `[1; 3; 2; 4]`. An equivalent result could be obtained by `Y)` or `Z)` without specifying any index (so that only `:` is applied): `[1 2; 3 4]1$Y)` or `[1 2; 3 4]1$Z)`.

`Y:` takes one input, which must be a cell array, and applies MATLAB's `{:}` reference indexing. This has the effect of "unboxing" the cell array, that is, it produces its contents (in column-major order), which are pushed onto the stack. The output specification for `Y:` controls how many or which of the input contents are pushed onto the stack; by default all contents are pushed. For example, `{{'Gödel' 'Escher' 'Bach'} 'Hofstadter' 1979} Y:` would consume the input cell array and push onto the stack, in this order: cell array `{'Gödel' 'Escher' 'Bach'}`, string `'Hofstadter'`, and number `1979`.

6 Control flow: conditional branches and loops

6.1 Conditional branches: "if…else"

An "if" branch is begun with `?`, ended by `]`, and can optionally contain `}`. These correspond to MATLAB's `if`, `end` and `else` respectively. `?` pops (consumes) the top array from the stack, and the statements until `}` or `]` are executed if the real part of that array has all nonzero elements[4]. Otherwise the statements between `}` and `]`, if any, are executed.

[4]This used to be MATLAB's behaviour in the past, if I recall correctly. Recent versions give an error when the expression has complex values.

6.2 Loops: “for”, “do twice”, “do... while” and “while”

A “for” loop is begun by `"` (double quotation symbol) and ended by `]`. The statement `"` takes (consumes) the top array from the stack. That element array is split into pieces, namely the columns of the array (or in general second-dimension slices, like with MATLAB’s `for`); and those pieces are iterated on (loop variable). In each iteration, all statements until `]` are executed. The current piece of the array can be obtained with the statement `@`; see below. The loop normally ends when all pieces have been iterated on. However, statements `.` (`break`) and `X.` (`continue`) can modify this behaviour, as will be discussed later.

A “do twice” loop is begun by `,` and ended by `]`. The statement `,` is equivalent to `[0 1]"`; that is, it produces a “for” loop with two iterations. The statements `.`, `X.` and `@` can be used as in a “for” loop. In particular, `@` will push `0` in the first iteration and `1` in the second.

A “do... while” loop is begun by `‘` (back tick) and ended by `]`. A “do... while” loop is different from MATLAB’s “while” loop in that the loop condition is evaluated at the end, not at the beginning. MATL initially executes all statements between `‘` and `]`. It then takes (consumes) the top array of the stack and checks if its real part has all nonzero elements (like MATLAB’s `while` does). If so it goes back to the beginning of the loop; else it exits the loop. As an example, `10‘1-t]` counts from `9` down to `0`. Statements `.` and `X.` can also be used in “do... while” loops. `@` in this type of loops pushes the current iteration index.

A “while” loop is begun by `X‘` and ended by `]`. In this case the loop condition is evaluated at the beginning, as happens with MATLAB’s `while` loops. Statements `.` and `X.` can also be used, and `@` pushes the current iteration index.

The statement `@` within a “for” or “do twice” loop pushes the current value of the loop variable, that is, the current part of the array that is being iterated on. Within a “do... while” or “while” loop it pushes the current iteration index, starting at 1. The output of `@` is that corresponding to the *innermost* loop.

The statement `X@` pushes the current iteration index of the innermost “for” loop, starting at 1. If `X@` is within a “do twice”, “do... while” or “while” loop, it still refers to the innermost “for” loop which contains it. That is, “do twice”, “do... while” and “while” loops are “transparent” for `X@`.

The statement `.` (“break”) causes the program to exit the innermost “for”, “do twice”, “do... while” or “while” loop.

The statement `X.` (“continue”) ends the current iteration of the innermost loop, and the loop proceeds with the next iteration, if any.

The statement `}` (“finally”) can optionally be used in “do... while” or “while” loops. If present, it causes the top of the stack to be consumed at that point and used as loop condition. The statements between `}` and `]` will be executed only at the end of the last iteration, before the loop is exited. That is, if the array used as loop condition has all elements with nonzero real part MATL goes back from that point to the beginning of the loop; else the statements between `}` and `]` are

executed and the loop is exited. In other words, `}` ends the loop body, but the code between `}` and `]` is executed at the end of the last iteration. This may be useful in order to get the last iteration index before exiting.

Statements `]` at the very end of the program can be omitted; see §7.

> I initially defined MATL with "for" and "while" loops only, as in MATLAB. But I realized that all "while" loops I was writing had a `T` before `‘` to force the loop code to be executed at least once. So I introduced "do...while" loops. The "finally" statement was a later addition.
>
> The "do twice" loop modifies the semantics of `@`, but not of `X@` (which will still refer to the minimally outer "for" loop). Another possibility would have been to also leave `@` unaffected by "do twice" loops, thus allowing access to the variable of an outer "for" loop. However, it is sometimes useful to have information about the iteration index of the "do twice" loop. In addition, having `@` push `0` in the first iteration and `1` in the second allows writing partially repeated code with the pattern `aabbaacc` (where `aa` etc represent different code parts) as `,aa@?cc}bb]`.

7 Implicit actions

7.1 Initial actions

When a MATL program is run, it begins with `tic`, `format compact`, `format long`, `warning('off','all')`, `close all`. The colormap is set as `gray(256)`. The seed of the random number generator is set using `rng('shuffle')`.

7.2 Final actions

At the end of the program, any loops or conditional branches that have not been closed by a corresponding `]` statement are implicitly closed. That is, as many `]` statements as needed are implicitly included after the last (explicit) statement in the source code.

After that, the display function `XD` is implicitly called. This function (see §8 and §A) by default takes each element from the stack, converts it to string if needed, and prints it on screen. Cell array inputs are unboxed and their contents are displayed in linear order. Nested cell arrays are allowed. For example, consider that the stack contains two elements: number `1` and the cell array `{'aa' {3 4; 5 6}}`. Then the displayed values will be, in this order: `1`, `aa`, `3`, `5`, `4`, `6`.

The number of inputs for the implicit display function can be modified by one final call to `$`. For example, to display only the top of the stack specify `1$` at the end of the program; or use `&`, which for `XD` has the same meaning as `1$` (see §5.2). To prevent the implicit display function from displaying any stack contents use `0$` (or delete all stack contents).

If called with 1 or more inputs, this function modifies clipboard M, and consumes its inputs from the stack, exactly as if the program included an explicit `XD` statement at the end. (This is probably irrelevant anyway, because it is by definition the last function call, so the clipboard or the stack can't be accessed afterwards).

> In fact, this implicit function is implemented by the compiler adding a `XD` statement at the end of the source code. This statement is marked as implicit for displaying purposes, but when generating the compiled code it is treated as if it were an explicit function call.

When the program finishes, the original MATLAB `warning` state and default colormap are restored.

7.3 Implicit inputs

In addition to the explicit `i` and `j` input functions (see §8), user input is implicitly triggered when a non-existent element of the stack is accessed by *any* statement. This implicit input is always *evaluated* (with content checking), that is, corresponds to function `i`. But the implicit input method is *not a function call*. It does not use or consume `$` or `#` specifications, and it doesn't affect clipboard M. It does produce an automatic copy of the input to (a new level of) clipboard G.

In general, the stack at any given moment contains elements at positions 1, 2, ... N, where N is the current number of elements in the stack (possibly 0). Implicit inputs can be viewed as follows: the stack is indefinitely *extended below the bottom*, that is, at positions 0, -1, -2, ... with values that are not initially defined, but are resolved on the fly via implicit input. These inputs are asked from the user only when they are needed, in the order in which they are needed. If several inputs are required at the same time they follow the normal stack order, that is, the input that is deepest in the (extended) stack is entered first.

A few examples will help clarify this. Consider the program `3^`. The function `^` needs two inputs but there is only one element in the stack (namely, number 3 at stack position 1). Thus MATL needs to "use" stack position 0 as well, which triggers user input. The number or array introduced by the user, say x, will be placed *below* the original 3. So the result will be x^3. To compute 3^x with implicit input, a swap operation would be needed: `3w^` (and then it is `w` that triggers the implicit input); or explicit input could be used: `3i^` .

The program `+^` asks for three numbers x, y, z in that order, and computes z^{x+y}. It works as follows. The stack is initially empty. When interpreting `+`, MATL asks the user for two numbers, x and y. So the stack now contains x and y, bottom to top. After `+` the stack contains a single value, $x+y$. Then `^` asks for another value, z, to fill the position below $x+y$. So the stack now contains z, $x+y$. After `^` it contains a single value, z^{x+y}. Therefore, the code is equivalent to `ii+iw^` (not to `iii+^`, as one might initially think).

The program `y` will implicitly ask for two inputs, and then push a copy of the first, leaving the stack with three elements. Note that even though `y` here copies only one element, that element is *two* positions below the current bottom of the (empty) stack, and consequently two implicit inputs are required.

A single input is implicitly asked from the user also when function `G` is called and clipboard G doesn't have any levels, as was described in §5.4.2.

> It would be nice to have implicit input of evaluated or unevaluated strings depending on the function. However, this seems difficult to do. For example, the second argument to `sum`, `rand` or `num2str` may be a number or a string, and it can't be told which one would be appropriate.

7.4 Automatic file input and output

If a file called `defin` exists when program execution is started, its contents are read in as bytes and converted to char type. A string (row vector of characters) containing all those values is pushed onto the stack before the first statement of the program is executed. (There is also a function for manually reading a file; see §8).

All screen output generated by the program is saved to a text file called `defout`. This file is overwritten if it previously existed. (There is also a function for manually writing to a file; see §8).

8 Table of functions

Table 4 shows the set of MATL functions, as well as control flow modifiers, separator and comment characters. The detailed definition of each function appears in Appendix §A.

> The table is under development. Functions may be added, removed, or changed.
>
> `e` and `j`, when used as a functions, may need a separator to isolate them from a preceding number.
>
> Similarly `:`, when used as a function, may need a separator to prevent it from being interpreted as part of a colon numerical vector.

Most MATL functions correspond to MATLAB functions. These have been selected based on perceived most common use. They include `@Divakar`'s list[5], and most functions from a list by Mathworks[6].

In some cases the MATLAB function has been extended or slightly modified. This is indicated in the function description of Appendix §A.

[5] http://stackoverflow.com/users/3293881/divakar

[6] http://es.mathworks.com/help/matlab/functionlist.html

All MATLAB functions supported by `bsxfun` (as well as some others) have automatic singleton expansion in MATL.

Most strings that are used as inputs to functions (for example, `'last'` option for `unique`) can be produced by calling one of the functions `X0`...`Z9` with a numeric input. Also, some functions allow using numbers instead of strings as input flags. See §A for details.

Table 5 summarizes function behaviour regarding their consumption of inputs and of input/output specifications, and their overwriting the function-input clipboard.

Some functions I didn't include, and the reasons why:

- `true`, `false` functions; they can be done with `zeros` and `ones` (which are included) followed by conversion to `logical` or by negation (each is just one character).
- `ipermute`: I don't think it's used often; and can be substituted by `permute`.
- `ndims`: it can be realized in two characters with `size` followed by `numel`.
- `isempty`: I removed it because it can be done easily with `numel` followed by logical negation.
- `length`. I followed `@chappjc`'s advice[7]: "Never use `length`. Ever."
- I initially thought it would be cool to have function equivalent to `eval` in MATL. But it would be hard to implement, and probably not very useful anyway.
- `cellstr`: function `mat2cell(x, ones(size(x,1),1), size(x,2), ..., size(x,ndims(x)))` exists. It's a generalization of `cell2str` that also works for numeric and logical values, and for any number of dimensions.
- `shiftdim`: it seems unnecessary with `permute`.
- `iscolumn`, `isvector` etc: they can be realized with `size`, and are not much used anyway, I think.
- `issorted`: it can be easily realized by comparing with the result of `sort`. Of course `issorted` would be more time-efficient, but time efficiency is not the main purpose of code golf.
- `rem`: having `mod` is probably enough.
- matrix inverse: not used very often; and can be done with matrix power, or with `eye` and matrix division.

[7] https://web.archive.org/web/20150915161950/http://stackexchange.com/users/3302774/chappjc

Table 4: Table of MATL statements

		x	y	z
	separator			
!	.' (transpose) / permute	rot90	system	full
"	for	repmat	repelem (run-length decoding)	blanks
#	specify outputs	display stack (debug)	sound, soundsc, audiowrite	fopen, fwrite, fclose
$	specify inputs	sym / str2sym	char(vpa(str2sym(…), ...))	fopen, fread, fclose
%	comment	class	cast	typecast
&	alternative default input/output spec	intersect	and	bitand
'	*Not used. String delimiter*	execute Matlab function	run-length encoding	now / clock
(	( ) assignment indexing	{ } assignment indexing	( ) assignment ind. with final :	( ) assignment ind. with initial :
)	( ) reference indexing / split	{ } reference indexing	( ) reference ind. with final : / split	( ) refererence ind. with initial : / split
*	.*	kron	matrix product	Cartesian product
+	+		conv2	conv2(…, 'same') / cconv
,	do twice	cos	sin	tan
-	-	setdiff	deconv	
.	break	continue	pause	bitget
/	/	angle	right matrix divide	unwrap
0	*Not used*	predefined literals	predefined literals	
1	*Not used*	predefined literals	predefined literals	
2	*Not used*	predefined literals	predefined literals	
3	*Not used*	predefined literals	predefined literals	
4	*Not used*	predefined literals	predefined literals	
5	*Not used*	predefined literals	predefined literals	
6	*Not used*	predefined literals	predefined literals	
7	*Not used*	predefined literals		
8	*Not used*	predefined literals		
9	*Not used*	predefined literals		
:	colon (range)	linearize array	comma-separated list	bitset
;		acos	asin	atan2
<	<	min	cummin	
=	==	isequal	strcmp	
>	>	max	cummax	
?	If		why	sparse
@	"for" / "do twice" value / "while" index	"for" index	perms	randperm
A	all	all(…, 1)	dec2base. Larger base, any symbols	base2dec. Larger base, any symbols
B	logical(dec2bin(...)-'0')	bin2dec(char(...+'0'))	dec2bin	bin2dec
C			im2col	im2col(…, 'distinct')
D	disp(num2str(…, ...)) / mat2str	disp(num2str(…))	sprintf / fprintf	disp
E	multiply by 2	replace elements in array		ifft, ifftn
F	*Not used. False (literal)*		exponents of prime factorization	fft, fftn
G	Paste from clipboard G (user-input)	plot	imwrite / imagesc / image / imshow	appearance of graphics / format
H	Paste from clipboard H	Copy to clipboard H	advanced plotting functions	
I	Paste from clipboard I	Copy to clipboard I	col2im	image processing functions
J	Paste from clipboard J	Copy to clipboard J		symbolic-specific functions
K	Paste from clipboard K	Copy to clipboard K		
L	Paste from clipboard L (multi-level)	Copy to clipboard L (multi-level)	gallery	
M	Paste from clipboard M (function-input)	mode		
N	stack size	nchoosek (array)	NaN	isnan
O	zeros	datestr	datenum	datevec
P	flip	flipud	pi	pdist2 / entries below diagonal
Q	increment by 1	accumarray	rat	polyval / roots / polyfit / inpolygon
R	triu	triu(…,1) / build matrix	tril	tril(…,-1) / build matrix
S	sort	sortrows	circshift	sign / fftshift / linspace
T	*Not used. True (literal)*		toeplitz	
U	str2num / string to array / square	str2double		
V	num2str			
W	2 raised to input			
X	*Not used*	regexp	regexprep	
Y	*Not used*		inf	isinf
Z	*Not used*			
[	*Not used. Array delimiter*	ind2sub		
\	mod	mod(…-1)+1	left matrix divide	divisors
]	end (loops or conditional branches)	sub2ind		
^	.^	sqrt	matrix power, or sum of matrix powers	Cartesian power
_	unary minus / normalize uint8			
`	do...while	while	tic	toc
a	any	any(…, 1)	padarray / unpad array	base2base
b	bubble		strsplit	
c	char (also for cell array)	cat	strcat	strjoin / convert to '#' and char 0
d	diff	diag / spdiags	blkdiag	gcd
e	reshape / squeeze		expm / logical "infinite" graph power	exp / Levenshtein distance
f	find	strfind	factor / divide by gcd	
g	logical / cell2mat	ndgrid	gamma / gammainc / betainc	gammaln / betaln
h	horzcat	{…, …}	hankel	hypergeom
i	input	urlread	imread	
j	input(…,'s')	real	imag	conj / real and imag
k	lower / floor	upper / ceil	closest values	
l	ones	clamp (limit to a range)	log. With two inputs, specifies base	log2
m	ismember	ismember(…,'rows')	mean	lcm
n	numel / size	nchoosek (numbers) / multinomial c.	poly / interp1	
o	double / cell array to numeric / parity	int64	round / change case	fix
p	prod	prod(…, 1, ...)	cumprod	isprime / totient function
q	decrement by 1	quantile	n-th prime / next prime	primes
r	rand	randn	randi	randsample / shuffle
s	sum	sum(…, 1, ...)	cumsum	std / cov / skewness / kurtosis
t	duplicate elements			strrep
u	unique	unique(,…'rows')		strjust
v	vertcat		eig / svd / strtrim	symmetric range / array / deblank
w	swap			
x	delete from stack	clc		
y	duplicate element	eye	hypot	size
z	nnz / cellfun(@nnz, ...)	nonzeros / remove whitespace		
{	*Not used. Cell array delimiter*	num2cell	mat2cell	mat2cell(x,ones(size(x,1),1),size(x,2))
|	abs / norm / determinant	union	or	bitor
}	else / finally			split array
~	Not	setxor	xor	bitxor

Normal function	Normal function: indexing	Normal function: literal	Meta-function	Stack rearranging function
Clipboard function	Control flow	Used only in literals	Separator, comment	Not used

Table 5: Behaviour of functions regarding consumption of inputs, consumption of input/output specifications and overwriting of function-input clipboard

	Consume inputs	Consume input/ output specifications	Overwrite function-input clipboard
Normal	Yes	Yes	Yes[a]
Meta	Yes	No	No
Stack rearranging	No	Yes	No
Clipboard	No	Yes	No

[a] If the function call has inputs

- `rank`, matrix pseudoinverse, matrix decompositions: probably too specialized.
- `isequaln`: not used very often, is it?
- `swapbytes`: too specialized. Or is it worth including it?
- `factorial`: it can be done with `:` and `prod`.
- `bitshift`, `bitcmp`: other bit-wise functions have been included. These two can be easily done with arithmetic operations.
- `strcmpi`: there's `strcmp`.
- `convmtx`: `convmtx(x,n)` can be realized as `conv2(x, eye(n))`.
- `histcounts` / `histc`: `histc` would be a little redundant; compare `histc(x,unique(x))` with `8#u` or `SY'`; or compare `histc(x,y)` with `!=s`. Besides, MATLAB's `histc` is deprecated and replaced by `histcounts`. The latter works differently; and I preferred the old `histc`'s behaviour: given `x = [10 20 30 20 30]`, `histcounts(x,unique(x))` gives `[1 4]` whereas `histc(x,unique(x))` gives `[1 2 2]`.

9 The MATL compiler

9.1 Usage

The official MATL compiler is written in MATLAB. It takes a source program in MATL and produces an output (compiled) program that is run in MATLAB. It requires version R105b or newer, although most of the functionality probably works in older versions too. The compiler also works on Octave 4.0 or newer; see §10. In addition, online compilers are available; see §11.

The compiler can be used in any of the following ways:

- `matl -options program`, or `matl -options 'program'` (command syntax);

- `matl('-options','program')` (function syntax).

In either case, the first input specifies processing options, starting with the character `-`; and the second input is a string or character array that contains the MATL program, or the name of a text file where the MATL program is stored. Both input arguments are optional.

The **first input** argument can specify the following options for the `matl` command:

- `p`: parse. Writes parsed program, with one line for each statement and using indenting, in text file `MATLp.txt`.

- `l`: listing. The parsed MATL program listing is shown on screen with one line for each statement and using indenting. Implies `p`.

  Two single-digit numeric options can be used to specify the indenting base (number of spaces to be included before any statement) and the indenting step (number of additional spaces for each nesting level). Default values are 4 and 2 respectively. If only one numeric option is provided, it is interpreted as the indenting base.

  If a *third* numeric option is provided, it is interpreted as the minimum number of spaces before comment symbol. A comment symbol is included at the end of each line, with all comment symbols vertically aligned and separated at least the specified number of spaces from the corresponding statement. This is useful for adding explanations to the code.

  Numeric options can be digits `0`, ..., `9`; or they can be *capital* letters `A`, `B`, ..., `Z`, which are interpreted as numbers 10 (`A`), 11 (`B`), ..., 35 (`Z`).

  File `MATLp.txt` containing the parsed program uses the indenting step, but not the indenting base.

  If options `c`, `r` or `d` have also been provided, the `matl` command stops at this point and waits for a key press before continuing, in order to give the user time to see or copy the displayed listing.

- `e`: listing with comments. It's like option `l` but automatic comment texts are added depending on the statement. These provide a good starting point to explain what the code does. Implicit statements (see §7) are also indicated, as comments only.

  In addition to the three numeric options used for option `l`, a *fourth* number can be specified (using capital letters 10, 11, ...). This is interpreted as the number of spaces between comment symbols and comment text. If less than four numeric options are provided, the rest take their default values. The third and fourth numeric options have default values 6 and 1 respectively.

  File `MATLp.txt` doesn't include comments.

As happens with `l`, if options `c`, `r` or `d` have been provided in addition to `e`, the `matl` command stops at this point and waits for a key press before continuing.

- `c`: compile. Produces a `.m` file called `MATLc.m` that can be run in MATLAB. Implies `p`.
- `r`: run. Runs the compiled program in MATLAB. Implies `p` and `c`. If an error occurs in the program, the error message includes a link to open the parsed MATL file at the line of the statement that caused the error.
- `d`: debug. Runs the compiled program in MATLAB in debug mode. Implies `p` and `c`. Breakpoints are set at the beginning of (the MATLAB code corresponding to) each MATL statement, to allow step-by-step execution. The variable editor is opened to show the MATL stack (variable `STACK`), input and output specifications (`S_IN` and `S_OUT`), and clipboard contents (`CB_H`, ..., `CB_L`; `CB_G`; `CB_M`).

  (For debugging see also function `X#` in Section 5.3).
- `f`: file. Indicates that the second input argument will not be MATL code, but the name of a file containing the code.
- `v`: verbose. Causes the compiler to provide detailed information about what it's doing.
- `h`: help. provides command-line help.
- `o`: online. This option is used in online compilers. It produces the following changes:
  - removes the default input prompts;
  - doesn't generate file `defout`;
  - forbids functions that read files or URL's;
  - forbids functions that write to files;
  - forbids functions that evaluate arbitrary MATLAB functions or system commands.

If no options are provided (or only option `f` is provided), the `matl` program defaults to `-r` (or `-rf`).

The **second input** argument is a string that represents one of the following, depending on the selected options:

1. The program code (options `p`, `l`, `e`, `c`, `r`, `d`);
2. The name of a file containing the program code (option `f`);

3. Search text to get help (option `h`).

If the program code, file name or search text have spaces or other symbols that cause MATLAB to misinterpret the string, it needs to be enclosed in quotation marks; and then quotation marks contained within the string need to be duplicated.

In cases 1 and 2 the second input argument may be omitted. In this event the `matl` command waits for input from the keyboard containing the MATL program or the file name. The MATLAB prompt is changed to a space followed by a single `>` symbol, which indicates MATL input mode. A program may be entered in a single line or in several lines (`Enter` key), and the end of the program is indicated by a blank line (`Enter` key twice). A file name is entered in a single line.

In case 3, if no string is provided as second input, general help about `matl` options is displayed. If a string is provided, it may contain the name of a MATL statement, or arbitrary search text. In the former case the information about that statement will be printed. In the latter, the search is based on case-insensitive partial matching with descriptions of MATL statements (which include equivalent MATLAB function names) and with automatic comment texts.

Examples:

- `matl`: waits for user input, and runs the program represented by that input. Files `MATLp.txt` and `MATLc.m` are produced.
- `matl '10:"@D']`: runs the program `10:"@D]`. Files `MATLp.txt` and `MATLc.m` are produced.
- `matl -d I4+`: runs the program `I4+` in debug mode (quotation marks are needed because the code contains a space). Files `MATLp.txt` and `MATLc.m` are produced.
- `matl -cf file.txt`: compiles the program contained in file `file.txt` and produces files `MATLp.txt` and `MATLc.m`.
- `matl('-l82v', myProgram)`: parses the program held in character array `myProgram`, shows the parsed result without automatic comments, using the numeric options for indenting, and produces file `MATLp.txt`. Detailed information about the process is also shown on screen.
- `matl -e0291 '1t8:"yy+'`: parses the program provided as string input (code for the Fibonacci sequence given in §1.3) and shows the parsed result with automatic comments, using the numeric options for indenting. The parsed file `MATLp.txt` is also produced. The result printed on screen is as shown in Figure 1.
- `matl -h ismember` would produce the result shown in Figure 2.

```
1         % number literal
t         % duplicate elements
8         % number literal
:         % range; vector of equally spaced values
"         % for
  y       % duplicate element
  y       % duplicate element
  +       % addition (element-wise, singleton expansion)
          % (implicit) end
          % (implicit) convert to string and display
```

Figure 1: Example of parsed code with automatic comments

```
m   true for set member
    2--4 (2);  1--2 (1)
    ismember. It also works for cell arrays with numeric content, or
    with mixed char/numeric content. See also Xm
Xm  true for set member, row-wise
    2--3 (2);  1--2 (1)
    ismember(..., 'rows', ...). See also m
```

Figure 2: Example of command-line help

9.2 Structure

The `matl` program consists of a **main function**, `matl`, which calls three other functions, corresponding to the parser, compiler, and runner/debugger.

The **parse function**, `matl_parse`, separates the program into statements. It includes the array checking referred to in page 7. Optionally it calls a display function, `matl_disp`, to print the listing of parsed code on screen. The output of the parser is a MATLAB struct array `S` in which each entry is a statement, with fields describing the source code of the statement, the statement type and other information useful for the compiler.

The **compile function**, `matl_compile`, generates MATLAB code corresponding to each statement: literals, functions and control flow statements.

Each function is defined by the MATLAB code that the compiler generates for that function. That code is divided into *preamble*, *function body* and *postamble*. Most of the preamble and postamble code is common to different groups of functions: take inputs from stack, push results onto stack, delete `$` and `#` specifications). The function body, as well as some parameters to be used in the preamble and postamble, are specific to each function, and define what the function actually does.

Functions are defined by means of a *function definition file*, `funDef.txt`. It

is a tab-separated plain text file that for each function contains the function body (directly as MATLAB code) and information that controls how the preamble and postamble code should be generated. This information includes allowed and default numbers of inputs and outputs, and whether the inputs should be consumed (deleted from the stack). The file also contains the text used in automatic comments.

One of the things the preamble does is to define how many outputs the function must produce (whether specified by a previous `#` statement or by default). The function is then responsible to follow that indication. The preamble code inserted by the compiler depends on the information obtained from the function definition file.

The maximum and minimum number of allowed outputs for each function are fixed numbers specified in the function definition file. For the default number of outputs there are three possibilities:

- Most commonly it is also a *fixed number* indicated in the function definition file. This number is read by the compiler to generate the appropriate preamble code.

- For some functions, the default number of outputs cannot be determined at compile-time. Consider for example the function `H`, which pastes the contents of clipboard H. Its default number of outputs is the number of elements contained by the clipboard, and thus can only be determined at run-time. In the function definition file this parameter is defined by a *string* that gets directly inserted into the compiled code, and will thus be run. The string can refer to the run-time information it needs, such as number of elements in the clipboard.

- For a small set of functions, the default number of outputs is very difficult to compute even at run-time before the function is actually executed. For example, the function `X)`, which applies `{...}` reference indexing, produces a default number of elements that depends in a complicated manner on the input array size and on the input indices (specially if `end`-based indexing is used). In this case the default number of elements will be determined after the function has finished its computations. This is indicated in the function definition file by a *negative* number as default number of outputs. This signals that it's the responsibility of the function at run-time to compute the default number of outputs and produce that many outputs, or to produce the specified number of outputs if it exists.

The postamble will discard some of the function outputs if the output specification is a logical array.

The function definition file is processed by a `genFunDef` function, which generates a *function definition struct array*, `F`, to be used by the compiler. This array

contains the same information as the text file, and is saved into a `funDef.mat` file for future use. When the compiler encounters a function in the MATL parsed code, it looks it up in array `F` and generates the compiled (MATLAB) code accordingly. To speed up compilation, the generation of struct array `F` is only done when the function definition file is found to be newer than the processed file `funDef.mat`; otherwise the latter is loaded.

For functions that generate predefined literals, these are specified on a separate *predefined literal file*, `preLit.txt`. It is a plain text file that for each function defines a set of key-value pairs. This is processed by a `genPreLit` function, which generates a *predefined literal struct array*, `L`, to be used by the compiler. This array contains the same information as the text file, and is saved into a `preLit.mat` file for future use.

Lines of compiled code are stored in a cell array of strings `C`, from which the compiled file `MATLc.m` will be written.

The function definition file and predefined literal file centralize all information about functions. This allows to define new functions without actually modifying the compiler code.

The **run function**, `matl_run`, executes the compiled program. If an error is found, an error message is issued with a link to the MATL statement that generated the error. In debug mode it inserts breakpoints and opens relevant variables (taking advantage of MATLAB's `openvar`).

The **help function**, `matl_help`, provides command-line help. It uses a struct array `H` that contains help information for all MATL functions and statements. This struct array is generated from the function definition file, the predefined literal file and additional information by a `genHelp` function, and is stored in file `help.mat`.

10 Compatibility with Octave

The compiler can also run on Octave, which is a free alternative to MATLAB. Version 4.0.0 or newer is required, although most of the functionality probably works in older versions too.

Although MATLAB and Octave have a large degree of similarity, compatibility between them is not total. This has some implications that are described in the following.

10.1 Compiler

The debug mode of the compiler uses MATLAB's `openvar` function to visualize the stack, clipboards and input/poutput specifications. This can't be done in Octave, because `openvar` doesn't exist.

Octave allows the " symbol as string delimiter, in addition to '. This implies that MATL programs that include " need to be enclosed in quotation marks, in addition to those that include '. Note that MATL only uses ' as string delimiter.

An internal aspect of the compiler is that Octave randomly initializes the random number generators automatically, so the compiled code doesn't need to do it.

10.2 MATL functions

The definitions of MATL functions are based on MATLAB. If there are differences between a MATLAB function and the corresponding Octave function, the corresponding MATL function is defined following that of MATLAB. This means that the compiler should reproduce that behaviour even when working on Octave, in order to achieve consistent results of MATL programs.

The choice of MATLAB over Octave, admittedly arbitrary, is based on the fact that the author has more experience with MATLAB, and by no means is meant to suggest any of them is better than the other. In any case, differences between MATLAB and Octave functions are few and small, so a programmer used to Octave will not have difficulties using MATL functions.

The compiler should ensure consistent behaviour of MATL programs regardless of the underlying platform being MATLAB or Octave. In the cases when inconsistencies between MATLAB and Octave have been identified, the compiler has been programmed to correct them, if feasible. The exceptions are cases where the correction would be too difficult or when the difference is unimportant (for example, because it occurs using a function with inputs for which behaviour is undocumented).

The way the compiler deals with differences between MATLAB and Octave is as follows. If an Octave function `foo` behaves differently from the corresponding MATLAB function, the compiled file `MATLc.m` in Octave, which is a function, includes a subfunction that intercepts Octave's `foo`. This subfunction applies the needed modifications, and usually includes a call to Octave's original `foo` via a `builtin` statement. With this approach, the definitions in the MATL function definition file (`funDef.txt`) are the same for MATLAB and for Octave. In other words, the definitions assume MATLAB, and any deviations that might be caused by Octave are dealt with by the compiler.

Similarly, if a MATLAB function doesn't exist in Octave, the compiler includes it as a subfunction in the compiled file.

These compatibility functions are defined in separate files in a *compatibility folder*, called `compatibility`. Each file is read as text by the compiler, if needed, to insert it as a subfunction in the compiled code.

The following list describes the cases where a discrepancy between a MATLAB and an Octave function has been detected, and indicates if the compiler

addresses this or not. In most cases Octave 4.0.0 is assumed. Some of these differences may not exist in newer Octave versions.

- `num2str` behaves differently in Octave and in MATLAB[8]. The compiler tries to achieve consistent behaviour, adapting the compiled code to give the same output on MATLAB and on Octave in the most common cases.

- `im2col` behaves differently in Octave and in MATLAB when the block size exceeds the array size. For example, `im2col(1:8, [2 1])` outputs `[]` in MATLAB, and gives an error in Octave.

  `im2col` allows char input in MATLAB, but not in Octave.

  Another difference exists regarding 3D input arrays. For example, the code `im2col(cat(3, magic(3), -magic(3)), [1 2])` gives different output in MATLAB and in Octave. Apparently MATLAB collapses all dimensions of the input array beyond the first.

  The above have been fixed. The compiler enforces MATLAB's behaviour when working in Octave.

- MATLAB's `im2col` ignores the third and subsequent components of the second input: `im2col(1:8, [1 2 3 4])` gives the same result as `im2col(1:8, [1 2 3 4])`. On the other hand, Octave gives an error.

  With the `'distinct'` option the behaviour is reversed: MATLAB gives an error, and Octave produces a result filled with zeros.

  These differences aren't addressed in the compiler, because they are cases that are probably outside the intended use of `im2col`.

- `spiral` doesn't exist in Octave. The compiler defines it as part of the compiled code if needed.

- `triu`, `tril` with `[]` and a nonzero number as inputs give an error in Octave. This has been corrected so that they return `[]` as in MATLAB.

- `unique` in Octave behaves differently than in recent MATLAB versions (it behaves like in old MATLAB versions). For the purposes of the compiler this function is modified so that it finds the first occurrence of each element, not the last; and second and third outputs are always column vectors.

  Support for the `'stable'` flag has been added.

- In `union`, `intesect`, `setdiff`, `setxor`: support for the `'stable'` flag has been added, only in the single-output case.

[8] http://stackoverflow.com/q/34483961/2586922

- The second output of `ismember` in Octave gives the index of the last occurrence, not the first as in recent versions of MATLAB. Also, it gives error when the first two inputs are char and numeric, whereas in MATLAB that works. Both issues are fixed.

  In Octave 4.0 and 4.2, `ismember` incorrectly treats complex values in inputs as their absolute values.[9] This has been fixed.

- `randsample` with scalar first input produces a row vector in Octave, instead of a column vector as in MATLAB. This has been fixed in the compiler, forcing a column vector in Octave in that case.

- Octave's `nchoosek`, unlike MATLAB's, requires first input to be numeric. This has been fixed, so that it can be a one-dimensional character array or cell array.

- Octave's `vpa` displays unwanted output. This hasn't been solved; however, it only happens in the first call to a symbolic function.

- When using Octave's `vpa`, conversion from symbolic to char looses precision. This is solved by converting the output of Octave's `vpa` to string using `pretty` (instead of `char`). This introduces unwanted whitespace in the beginning, middle and end of the string (example: `pretty(vpa(1, 100))`). There may also be wanted whitespace separating elements of an array (example: `pretty(vpa(pi:5, 100))`) or with complex values. It is difficult to distinguish wanted space from unwanted one. So, for the scalar, real-valued case all whitespace is removed (because all is unwanted). In any other case the whitespace is not removed (because some is wanted).

  Also, in Octave trailing decimals equal to zero are produced. This has been fixed for real numbers.

- Octave's `vpa` seems to have trouble with long strings containing non-integer numbers. For example, compare the output of `vpa('1/1234567891011')` with that of `vpa('1/.1234567891011')`. This isn't addressed by the compiler.

- MATLAB does not support char input to `vpa` anymore. Consequently, `vpa(str2sym(...), ...)` is needed for char input. On the other hand, Octave does not have `str2sym`, but `vpa` allows char input representing a scalar, or `sym` can covert char input representing a scalar. The second option is chosen. The compiler uses `vpa(str2sym(...), ...)` for char input in the compiled code, but defines `str2sym` to call `sym` in Octave.

[9] https://savannah.gnu.org/bugs/index.php?52437

- Octave does not have `str2sym`. Conversion from string to symbolic is done using `sym` instead. However, this seems to only handle strings representing scalars.
- `sum` and `mean` don't support the `'omitnan'` flag in Octave. This has been added.
- `diff` and `mod` don't allow char inputs in Octave. This has been fixed.
- Null assignment with less indices than dimensions works differently in MATLAB and in Octave. The code `x = cat(3,[1 2; 3 4],[1 2; 3 4]); x(:,2) = [];` produces the result `[1 1 2; 3 3 4]` in MATLAB (trailing dimensions are collapsed), but gives `cat(3, [1;3], [1;3])` in Octave (no collapsing). This difference isn't addressed by the compiler.
- `dec2bin` and `dec2base` don't allow empty inputs or char inputs in Octave. This has been fixed.
- `hypergeom` doesn't exist in Octave. This is partly addressed by the compiler. The function is computed either by direct sum (when some of the numerator coefficients is a non-positive integer, in which case the sum is finite) or using a third-party function[10], which only works for certain ranges of input values.
- Octave's `disp`, unlike MATLAB's, produces some output for empty input. This has been fixed. Correcting this requires intercepting `disp` in Octave. This prevents the `disp` function for `sym` values from being called[11]. To solve this, the code of `disp` for `sym` values is manually included. This is not an elegant solution, but as a side effect it allows some tweaking, namely removing left indentation which is introduced by Octave and not by MATLAB.
- Octave's `str2func` works in the form `y = str2func('max')`, but not in the form `y = str2func('@(x) max(x)')`, unlike MATLAB's. This has been fixed. (This function is necessary for MATL's equivalent of `accumarray`.)
- In Octave arrays of type `char` cannot be converted to `logical`. This has been fixed.
- The three-input version of `circshift` doesn't exist in Octave. Also, Octave requires, unlike MATLAB, that the number of elements of the second input doesn't exceed the number of dimensions of the first. This has been fixed.

[10] http://www.mathworks.com/matlabcentral/fileexchange/5616-generalized-hypergeometric-function

[11] At least I don't know how to do it. See https://stackoverflow.com/questions/58727452/how-to-call-a-shadowed-function-in-octave

- In Octave, `pdist2` allows fewer distance functions than `pdist` does. This is solved (not very efficiently) by calling `pdist`, if needed, with the vertically concatenated inputs, and from the result extracting the desired values.

- `max` and `min` don't allow first and second inputs of type char in Octave. This has been solved.

- Some functions in MATLAB automatically convert input to `double` if needed, and in Octave don't. This has been solved. These functions include `cumsum`, `cumprod`, `kron`.

- Octave's `datestr` with input format `16` (`'HH:MM PM'`) produced hour numbers greater than `12`. This was initially corrected. Then an Octave patch[12] came out that corrects a different issue related to rounding, and that apparently incorporates the correction for input format `16` too. So the function from that patch is used.

- MATLAB's `regexp` and `regexprep` produce empty outputs or do nothing, respectively, if the regular expression is invalid; whereas Octave's give an error. This has been solved.

- MATLAB's `regexprep` accepts a numeric value as fourth input, whereas Octave's doesn't. This is partly addressed as follows: if fourth input is equal to 1 in Octave it is replaced by `'once'`, so that at least that one works.

- Octave's `strsplit` gives an error when trying to split on `+`. This has been fixed.

- Octave's `str2num` gives an error with certain inputs. This has been fixed.

- In Octave, `str2num` with input `'e'` produces number e (and similarly for inputs like `'[1; e]'`). In MATLAB it produces an empty array, that is, the string is not recognized as a number. This difference is addressed by the compiler, so that the behaviour in Octave is as in MATLAB.

- In MATLAB, `str2double` linearizes char input into a row, whereas in Octave it doesn't. This has been solved.

- MATLAB's `imshow`, by default, produces one screen pixel for each image pixel. On the other hand, Octave's `imshow` scales the image on the screen. This has been partly fixed, so image pixels and screen pixels coincide. "Partly" means that the fix only works in some platforms, for reasons unknown (at least by me).

[12] https://savannah.gnu.org/bugs/index.php?50673

- Octave's `mat2str` doesn't handle char input. This has been fixed for inputs containing printable ASCII chars.
- Octave's `strcat` doesn't remove trailing space when the input is a single string. This has been fixed.
- `y = [1 3 5; 2 4 6]; y([1 3 5]) = []` gives `[2 4 6]` in MATLAB but `[2; 4; 6]` in Octave. The pattern seems to be: if `~isvector(y)` MATLAB reshapes as a row, whereas Octave reshapes as a column. This is addressed by explicitly reshaping as a row in that case.
- Matlab's `disp` displays `char(0)` as space on any platform. Octave displays it as either space, or narrow space, or nothing.[13] MATL's display functions replace `char(0)` (and other chars with codepoint below 32) by space before calling `disp`.
- In MATLAB, `input(..., 's')` (unevaluated input) interprets input consisting of only spaces as empty input. This doesn't happen in Octave. This difference has not been addressed.
- In MATLAB, `sprintf('%1.1g', 0.25)` gives `0.3` on Windows (round away from zero) and `0.2` on Linux (banker's rounding). In Octave it gives `0.2` both on Windows and on Linux.[14] These differences have not been addressed.
- `char` implicitly rounds down in MATLAB, and rounds to closest in Octave. This difference has been addressed by enforcing MATLAB's behaviour. Octave's `mode` doesn't handle char input. This has been fixed.
- Octave treats ranges specially to save memory[15]. When applying function `nnz` to a range an error occurs. This has been fixed.
- `factor` applied to a symbolic scalar gives a symbolic array with the factors in MATLAB, but something different in Octave. This has been fixed.
- Octave's `reshape` for symbolic arrays doesn't allow `[]` as part of the shape specification. This hasn't been fixed.
- Octave's `ismember`, `mean`, `svd`, `gcd`, `lcm` don't work with symbolic data. This hasn't been fixed.
- `sum([],1)` gives an empty array of size 1×0 in MATLAB, but `0` in Octave. An analogous difference exists with `prod`. This has been fixed.

[13] https://chat.stackoverflow.com/transcript/message/29732017#29732017
[14] https://savannah.gnu.org/bugs/index.php?51515
[15] https://www.gnu.org/software/octave/doc/v4.2.0/Ranges.html

- `sum(sym([]))` gives `0` in MATLAB, but an empty array of size 1×0 in Octave. An analogous difference exists with `prod`. This has been fixed.
- `bwlabeln` with first input of size 1×1 different than `0` or `false` gives an error on some systems. This has been fixed.
- `perms` with an input array with more than one singleton dimension gives a different result in Octave than in MATLAB (where the result corresponds to linearizing the input). This has been fixed.
- `bwselect` in some versions of Octave's Image Package (for example 2.12.0) gives different results from those in Matlab. This has been fixed.

11 Running MATL online

MATL code can be executed online in the following two awesome platforms. Many thanks to `@Dennis` and `@Suever` for providing them!

- `@Dennis`' *Try It Online* platform[16]. The online compiler runs on Octave. Text output is displayed normally, and graphical output is displayed (with limitations) by Octave's fancy ASCII-graph capabilities.
- `@Suever`'s *MATL Online* interpreter[17]. This platform is MATL-specific and allows real-time text output, graphical output and documentation search, among other features.

12 Example programs explained

In the following, the examples given in §1.3 are explained. This serves to illustrate some of the features of MATL. See §8 and §A for the detailed definition of the functions involved.

12.1 Example 1: infinite loop

Code: `'T]`.

This is a "do-while" loop. This type of loop is always entered at least once. Within the loop, `T` pushes a `true` literal to make sure the loop condition is met. This literal is consumed and the program proceeds with the next iteration, indefinitely.

The code could be shortened to `'T`, because MATL automatically closes loops at the end of the program if they have not been previously closed.

[16] https://tio.run/#matl

[17] https://matl.io

12.2 Example 2: first 10 Fibonacci numbers

Code: `1t8:"yy+`.

The first statement, `1`, pushes number 1 onto the stack. `t` makes a copy of it, so the stack now contains `1`, `1`. Statement `8` pushes an 8. Function `:` by default takes one input, and transforms the top of the stack, which is `8`, into vector `[1 2 3 4 5 6 7 8]`. So the stack now contains, bottom to top: `1`, `1`, `[1 2 3 4 5 6 7 8]`.

Statement `"` begins a "for" loop applied on vector `[1 2 3 4 5 6 7 8]`. Thus there will be 8 iterations, each corresponding to an entry of this vector. Since the vector is consumed by `"`, at the beginning of the first iteration the stack contains `1`, `1`. The loop body begins with `y`, which duplicates the second-top element from the stack and pushes the copy onto the top. So `yy` duplicates the top two elements (like `2$t` would, but saving one character). Then `+` computes their sum to produce `2`.

Note that the `]` that closes the `"` loop is missing. This is allowed, because it is implicitly inserted by MATL at the end of the program.

So at the end of the first iteration the stack contains `1`, `1`, `2`. The second iteration again operates on the top 2 elements, `1`, `2`, to produce their sum `3`; and so on. The initial `1`, `1` remain in the stack as the first 2 members of the Fibonacci sequence, and the 8 loop iterations produce the next 8 Fibonacci numbers, to complete the total of 10.

At the end of the program, the function `XD` is implicitly called. By default this function displays all elements in the stack, from bottom to top.

12.3 Example 3: unique characters from string in original order

Code: `ju`.

`j` inputs a string (without any prompt string by default). The second statement is function `u`, which corresponds to `unique` with the `'stable'` flag by default. This function takes as input the string typed by the user to produce the desired result, which is displayed by the implicit final call to `XD`.

12.4 Example 4: unique characters from string in original order, manual approach

Code: `jtt!=XRa~)`. This produces the same result as in the previous example, but without using `u`.

`j` inputs a string. This is duplicated twice by `tt`, and the last copy is transposed by `!`. Function `=` consumes both copies (normal and transposed) and compares them element-wise with singleton expansion, producing a logical matrix. Entry

(m,n) of this matrix is `true` if and only if the m-th character of the input string equals the n-th character. Thus the matrix is symmetric and contains `true` values along the diagonal. The stack now contains the original string and the comparison matrix.

`XR` corresponds to `triu(..., 1)`, and thus keeps the part of the matrix above the diagonal, making all other entries `false`. This will assure that only duplicates with respect to *preceding characters* are detected.

`a` corresponds to `any`, with 1 input by default. The result is a logical row vector with a `true` at entry m if and only if the upper-triangular matrix produced by the previous function (`XR`) has at least a `true` value at column m; that is, if that character was a duplicate of a preceding character. Function `~` negates this logical vector, so that `true` indicates characters that should be kept. The stack now contains the original string and this logical vector. Function `)`, which takes 2 inputs by default, indexes the string with the logical vector to produce the desired result, which will be implicitly displayed.

13 Acknowledgments

Thanks to the following people (in alphabetical order) for their contributions:

- `@AnderBiguri` for his suggestion to use Octave's two-output version of `factor`, which was key to solving an Octave compatibility issue with this function applied to symbolic input.
- `@AndrasDeak` for his very extensive testing of the compiler. Also for his help (together with `@beaker`) regarding a bug in Octave's `ismember` function with complex inputs.
- `@beaker` for a helpful discussion that led me to reconsider including a form of `end`-based indexing, for telling me about Octave's `repelems` function, and for pointing out several errata. Also for his help (together with `@AndrasDeak`) regarding a bug in Octave's `ismember` function with complex inputs.
- `Ben Barrowes` for his function `genHyper`.
- `Conor O'Brien` for his suggestion to have a function essentially equivalent to `35*c` but shorter.
- `@David` for finding a good implementation of the "n-th prime" function, as well as some errata in this document. Also for suggesting that `G` should take implicit input if needed, and for finding a bug in Octave's `datestr` function.
- `@Dennis` for hosting MATL in his wonderful *Try It Online* platform[18]. Also

[18] http://tryitonline.net

for suggesting modular indexing, and bit-wise operations with negative values.

- `@DJMcMayhem` for finding some errata, and also for suggesting that a square-power and a divisors function be added.
- `@EriktheOutgolfer` for finding some errata in the documentation.
- `@excaza` for his suggestion regarding the hypergeometric function.
- `@flawr` for his suggestions, including the "n-th prime" function, "shuffle" function and implicit input of data, and for finding several bugs.
- `@FryAmTheEggman` for helpful comments that led to improvements related to `nchoosek`.
- `@Giuseppe` for finding a bug in `u`.
- `@H.PWiz` for correcting a typo in the documentation.
- `@pragmatist1` for suggesting the use of a regular expression to check contents of array literals.
- `@quartata` for his suggestions related to the determinant function.
- `@RainerP` for a helpful discussion related to "if" branches and the automatic function clipboard.
- `@rayryeng` for coming up with the initial idea[19] and for his invaluable help in getting MATL to work online. Also for his suggestion to include predefined constants, for reading an early version of this document and suggesting changes, and for his support with this project.
- `@Sanchises` for noticing bugs in `Za` (base conversion), in `Y@` (variations without repetition), in the Octave compatibility function for `X~` (`setxor`), the latter of which also affected other functions, and for finding another problem in that function, which turned out to be related to a weird Octave bug with vertical concatenation; for noticing a difference with `char`'s implicit rounding in MATLAB and in Octave; for suggesting to include `expm` (matrix exponential) and to generalize matrix power as a sum over different exponents; for finding a bug in the parser, which incorrectly identified `]` or `}` within strings as closing symbols for array literals; for suggesting to generalize `z` for cell arrays; and for suggesting that MATL should have a symbolic data type.

[19] http://chat.stackoverflow.com/transcript/message/25584017#25584017

- `@Steadybox` for finding a bug in Octave's `nnz` function when applied to a range.

- `@Suever` for his help in finding several differences between Octave and MATLAB, and for finding a bug in MATL's extension of `reshape`, in `YG` (`imshow`), and in Octave's compatibility patch for `u` and `Xu` (`unique`). Also for suggesting that `&` should produce a "secondary default" input/output specification, adapted to each function (my initial idea was to simply have it increase the number of inputs by 1), and for his valuable help in finding the most appropriate secondary specification for each function, including a very useful script to get information from MATL answers posted in the Programming Puzzles and Code Golf site. Thanks too for his suggestion to have automatic input transposition for single-input versions of addition, multiplication and other functions, and for his suggestion which led to the name "element-wise indexing". And, specially, thanks for creating and maintaining the awesome *MATL Online* platform[20].

- `@sundar` for prompting me to clarify that strings are understood as row vectors of chars in MATL; for several corrections in the documentation, for a correction in `Z{`; for a correction in `S`; for finding a bug in `X.`; and for suggesting the `Zx` function, two-input char mode for `Yo` and a modification in `Yb`.

- @TasosPapastylianou for pointing out that a shadowed function `disp(x)` for symbolic `x` can be called as `builtin('@sym/disp', x)` in Octave (`builtin('disp', x)` doesn't work).

Appendix A Detailed function definitions

MATL functions are defined in Table 6.

The notation for allowed, default and alternative numbers of inputs and outputs is as follows. Consider for example the `p` function (which corresponds to MATLAB's `prod`). Then "`p` 1–3 (1 / 2) 1" means that this function can accept 1 to 3 inputs, with 1 as default and 2 as secondary default; and produces 1 output. This notation may be abbreviated if the number of inputs or outputs is unbounded or is fixed: "`(` 3– (3 / 4) 1" means that the number of inputs can be any integer starting at 3, the number of outputs is always 1; the default is 3 inputs, and the secondary default is 4 inputs.

Table 6: Function definitions

[20] `https://matl.io`

`!`	1–2 (1 / 2)	1	If 1 input: `.'` (`transpose`), or `permute(..., [2 1 ...])` for multidimensional arrrays. If 2 inputs: `permute`. If the second input is 1, 2 or 3 it indicates which dimension is not permuted for a 3D array; that is, it corresponds to [1 3 2], [3 2 1] or [2 1 3] respectively
`X!`	1–2 (2)	1	`rot90`
`Y!`	1	0–2 (2)	`system`
`Z!`	1	1	`full`
`X"`	2– (3)	1	`repmat`
`Y"`	2– (2 / 3)	1	`repelem` (run-length decoding). With 2 inputs: if the first or second inputs are empty the output is empty. Else, the second input is linearized if needed; and if the first or second input is shorter than the other, entries are repeated cyclically to match lengths
`Z"`	1	1	`blanks`. It allows vector input, and in that case it produces a matrix or multidimensional array
`X#`	0	0	display stack as a cell array
`Y#`	2– (2 / 3)	0	(i) With 2 inputs: `sound`. If second input is `T`, it is interpreted as `44100`. If second input is `F`, it is interpreted as `44100` and `audiowrite` is called instead of `sound`, with file name `'audio.wav'`. (ii) With 3 inputs: if third input is a truthy scalar, `soundsc` is called with the first two inputs. If third input is a numeric vector of size 2, `soundsc` is called with the three inputs. In both cases, `T` in second input is interpreted as `44100`. If third input is a string, it defines a file name, and `audiowrite` is called with that file name and the other two inputs. (iii) With more than 3 inputs: `audiowrite` is called using the third input as file name. Further inputs specify parameter-value pairs. `T` in second input is interpreted as `44100`.
`Z#`	1–3 (1 / 2)	0	Writes first input to file `inout`, creating it if necessary. If the file exists, by default its previous contents are overwritten. If the input is a numeric or char array each entry is treated as a raw byte and written. Behaviour is undefined for entries exceeding 255. If the input is a cell array, the contents of each cell are written, with a byte 10 (representing newline) in between. With 2 inputs: second input specifies file name; it is converted to char is necessary; if empty defaults to `inout`. With 3 inputs: third input specifies whether previous contents of the file should be kept

`X$`	1	1	(i) For non-char input: `sym`. (ii) For char input: `str2sym`. *See also* `Y$`
`Y$`	2	1	(i) `char(vpa(str2sym(...), ...))` for char input, or `char(vpa(...))` for non-char input. Second input is number of significant digits. (ii) If the first input is a number or a string representing a real or imaginary scalar and the second input (number of significant digits) is negative, the latter is interpreted as its absolute value, and the output will give the actual digits without rounding (so inputs `'5/3'` and `-4` will produce `'1.666'`, not `'1.667'`). *See also* `X$`
`Z$`	0–1 (0 / 1)	1	Reads bytes from specifed file. Each individual byte is then converted to a char, and the output is a row vector of char. If 0 inputs or empty input: file name is `inout`.
`X%`	1	1	class of input (`class` with one input)
`Y%`	2–3 (2)	1	`cast`. This function allows strings in second input to be replaced by numbers, as follows: 1: `'uint8'`, 2: `'int8'`, 3: `'uint64'`, 4: `'int64'`, 5: `'uint16'`, 6: `'int16'`, 7: `'uint32'`, 8: `'int32'`, 9: `'double'`, 10: `'single'`
`Z%`	2	1	`typecast`. This function allows strings in second input to be replaced by numbers, as follows: 1: `'uint8'`, 2: `'int8'`, 3: `'uint64'`, 4: `'int64'`, 5: `'uint16'`, 6: `'int16'`, 7: `'uint32'`, 8: `'int32'`, 9: `'double'`, 10: `'single'`
`X&`	2–4 (2 / 3)	1–3 (1)	`intersect`. Uses the `'stable'` flag by default. If one input is char and the other is numeric, the latter is converted to char. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'rows'`, 2: `'stable'`, 3: `'sorted'`
`Y&`	0– (2 / ‡)	1	`&` (`and`), element-wise with singleton expansion
`Z&`	2–3 (2)	1	`bitand` (bitwise 'and'), element-wise with singleton expansion. If first or second inputs are `char` they are converted to `double`. `double` inputs are rounded. If the inputs are `double` they can have negative values, and in that case they must be in the range from `-2^52` up to `2^52-1`. *See also* `Z|`, `Z~`
`X'`	1– (2 / 2)	0– (1)	execute Matlab function specified by last input, using the rest of the inputs as arguments.
`Y'`	1	2 (2 / 2nd)	run-length encoding (inverse of `repelem`). Input may be an array or cell array. Numeric values must be finite

`Z'`	0–1 (0 / 1)	1	`now`. With 1 input: the input should be numeric with values from 1 to 6, which are used as indices into the output of `clock`
`(`	3– (3 / 4)	1	assignment `( )` indexing. Null assignment (`x(...) = []`) can only be done with a single index. *See also* `Y(`, `Z(`
`X(`	3– (3)	1	assignment `{ }` indexing
`Y(`	2– (3)	1	assignment `(..., :)` indexing. Null assignment (`x(..., :) = []`) can only be done with a single index (in addition to the implicit colon). *See also* `(`, `Z(`
`Z(`	2– (3)	1	assignment `(:, ...)` indexing. Null assignment (`x(:, ...) = []`) can only be done with a single index (in addition to the implicit colon). *See also* `(`, `Y(`
`)`	2– (2)	1–2 (1 / 2)	reference `( )` indexing. If 2 outputs: only one input index can be used. The second output produces the "complementary" array `y=x; y(ind)=[]`, where `x` and `ind` are the inputs. *See also* `Y)`, `Z)`
`X)`	2– (2)	0– (▽)	reference `{ }` indexing
`Y)`	1– (2)	1–2 (1 / 2)	reference `(..., :)` indexing. If 2 outputs: only one input index can be used. The second output produces the "complementary" array `y=x; y(ind,:)=[]`, where `x` and `ind` are the inputs. *See also* `)`, `Z)`
`Z)`	1– (2)	1–2 (1 / 2)	reference `(:, ...)` indexing. If 2 outputs: only one input index can be used. The second output produces the "complementary" array `y=x; y(:,ind)=[]`, where `x` and `ind` are the inputs. *See also* `)`, `Y)`
`*`	1– (2 / 1)	1	`.*` (`times`), element-wise with singleton expansion. If 1 input: a second input is used given by the first transposed
`X*`	2	1	`kron`
`Y*`	2	1	matrix product, `*` (`mtimes`)
`Z*`	1– (2 / ‡)	1	Cartesian product. Given a number *n* of arrays of possibly different sizes, generates an *n*-column matrix whose rows describe all combinations of elements taken from those arrays
`+`	1– (2 / 1)	1	`+` (`plus`), element-wise with singleton expansion. If 1 input: a second input is used given by the first transposed
`Y+`	2–3 (2 / 3)	1	`conv2`. Doesn't allow two-vector and one matrix mode. Converts first two inputs to `double`. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'same'`, 2: `'valid'`, 3: `'full'`. *See also* `Z+`

`Z+`	2–3 (2 / 3)	1	(i) If 2 inputs: `conv2(..., 'same')`. (ii) If 3 inputs: `cconv`. Non-vector inputs are linearized into column vectors. The output is of type `double`. Integer inputs are guaranteed to give exact integer results, up to `double` data type limitations. (i,ii) Inputs are converted to `double`. *See also* `Y+`
`X,`	1	1	`cos`
`Y,`	1	1	`sin`
`Z,`	1	1	`tan`
`-`	1–2 (2 / 1)	1	`-` (`minus`), element-wise with singleton expansion. If 1 input: a second input is used given by the first transposed
`X-`	2–4 (2 / 3)	1–2 (1)	`setdiff`. Uses the `'stable'` flag by default. If one input is char and the other is numeric, the latter is converted to char. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'rows'`, 2: `'stable'`, 3: `'sorted'`. *See also* `X~`
`Y-`	2	1–2 (1 / 2)	`deconv`
`Y.`	0–1 (1)	0	`pause` (without outputs)
`Z.`	2–3 (2)	1	`bitget`. If first input is `char` it is automatically converted to `double`
`/`	2	1	`./` (`rdivide`), element-wise with singleton expansion
`X/`	1	1	`angle`
`Y/`	2	1	right matrix division, `/` (`mrdivide`)
`Z/`	1–3 (1)	1	`unwrap`
`X0`	1	1	predefined literal depending on input
`Y0`	1	1	predefined literal depending on input
`X1`	1	1	predefined literal depending on input
`Y1`	1	1	predefined literal depending on input
`X2`	1	1	predefined literal depending on input
`Y2`	1	1	predefined literal depending on input
`X3`	1	1	predefined literal depending on input
`Y3`	1	1	predefined literal depending on input
`X4`	1	1	predefined literal depending on input
`Y4`	1	1	predefined literal depending on input
`X5`	1	1	predefined literal depending on input
`Y5`	1	1	predefined literal depending on input
`X6`	1	1	predefined literal depending on input

`Y6`	1	1	predefined literal depending on input
`X7`	1	1	predefined literal depending on input
`X8`	1	1	predefined literal depending on input
`X9`	1	1	predefined literal depending on input
`:`	1–3 (1 / 2)	1	`colon` (with three inputs `x`, `y`, `z` produces `x:y:z`; with two inputs `x`, `y` produces `x:y`). If one input: produces `1:x`, or `' ':x` if `x` is char. For a single cell-array input, the contents of the cell array are interpreted as the actual inputs.
`X:`	1	1	linearize to column array (index with `(:)`)
`Y:`	1	0– (△)	generate comma-separated list from cell array (index with `{:}`) and push each element onto stack
`Z:`	2–4 (3)	1	`bitset`. If first input is `char` it is automatically converted to `double`
`X;`	1	1	`acos`
`Y;`	1	1	`asin`
`Z;`	2	1	`atan2`, element-wise with singleton expansion
`<`	1–2 (2 / 1)	1	`<` (`lt`), element-wise with singleton expansion. For complex values, the real parts are compared. If 1 input: a second input is used given by the first transposed
`X<`	1–3 (1)	1–2 (1 / 2nd)	`min`. If 2 inputs: element-wise with singleton expansion. With more than 2 inputs, the second is replaced by `[]`. This function does not support flags `'omitnan'` or `'includenan'`. Output is `char` if there is 1 input and it is `char`, or if there are 2 inputs and both are `char`, or if there are 3 inputs, second is empty and third is `char`. *See also* `X>`, `Xl`
`Y<`	1–3 (1 / 2)	1	`cummin`. Output is `char` if first input is. *See also* `Y>`
`=`	1–2 (2 / 1)	1	`==` (`eq`), element-wise with singleton expansion. If 1 input: a second input is used given by the first transposed
`X=`	0– (2 / ‡)	1	`isequal`. Works for any number of inputs
`Y=`	2	1	`strcmp`. If first or second inputs are numeric they are converted to char
`>`	1–2 (2 / 1)	1	`>` (`gt`), element-wise with singleton expansion. For complex values, the real parts are compared. If 1 input: a second input is used given by the first transposed

`X>`	1–3 (1)	1–2 (1 / 2nd)	`max`. If 2 inputs: element-wise with singleton expansion. With more than 2 inputs, the second is replaced by `[]`. This function does not support flags `'omitnan'` or `'includenan'`. Output is `char` if there is 1 input and it is `char`, or if there are 2 inputs and both are `char`, or if there are 3 inputs, second is empty and third is `char`. *See also* `X<`, `Xl`
`Y>`	1–3 (1 / 2)	1	`cummax`. Output is `char` if first input is. *See also* `Y<`
`Y?`	0	1	answer why. Sort of
`Z?`	1–6 (3)	1	`sparse`. If 3 inputs and third input is `char`, the output is converted to `char`
`Y@`	1–2 (1 / 2)	1	If 1 input: `perms`. If 2 inputs: variations (without repetition). In either case, the rows in the result are sorted in ascending order as a group
`Z@`	1–3 (1 / 2)	1	`randperm` (produces a row vector as output). If 3 inputs: third input indicates number of permutations, each on a different row. If first input is char it is interpreted as population (not as size). *See also* `Zr`
`A`	1–2 (1 / 2)	1	`all`. *See also* `XA`
`XA`	1	1	`all(..., 1)`. *See also* `A`
`YA`	2–3 (2 / 3)	1	(i) `dec2base`. (ii) If second input has more than one element: it defines the symbols, which can be characters or numbers. The number of symbols defines the base, which can exceed 36. (iii) If second input is a negative number `-n`: it is interpreted as symbols `0:n-1` (case ii). (i, ii) Base `0` is interpreted as `10`, `1` as `16`, `F` as `0:9`, `T` as `0:15`. *See also* `ZA`, `Za`
`ZA`	2	1	(i) `base2dec`. (ii) If second input has more than one element: it defines the symbols, which can be characters (case-sensitive) or numbers. The number of symbols defines the base, which can exceed 36. (iii) If second input is a negative number `-n`: it is interpreted as symbols `0:n-1` (case ii). (i, ii, iii) Non-recognized digits are ignored. Base `0` is interpreted as `10`, `1` as `16`, `F` as `0:9`, `T` as `0:15`. *See also* `YA`, `Za`
`B`	1–2 (1 / 2)	1	`logical(dec2bin(...)-'0')`. *See also* `YB`
`XB`	1	1	`bin2dec(char(logical(...)+'0'))`. Works also for cell array input. *See also* `ZB`
`YB`	1–2 (1 / 2)	1	`dec2bin`. *See also* `B`

`ZB`	1	1	`bin2dec`. *See also* `XB`
`YC`	2–4 (2)	1	`im2col`. If the second input is a scalar `n`, it is transformed into `[1 n]` if the first input is a row vector, or to `[n 1]` otherwise. First input can also be a cell array. *See also* `ZC`
`ZC`	2–3 (2)	1	`im2col(..., 'distinct')`. If the second input is a scalar n, it is transformed into [1 n] if the first input is a row vector, or to [n 1] otherwise. First input can also be a cell array. *See also* `YC`
`D`	0– (1)	0–1 (0 / 1)	(i) With 0 outputs: If 1 input: `disp(num2str(..., '%.15g '))`. If several inputs: `disp(num2str(eachInput,lastInput))`, where `eachInput` loops over all inputs but the last. In either case, (nested) cell arrays are (recursively) unboxed in linear order. Most characters below 32 are replaced by space. Symbolic data is displayed directly. (ii) With 1 output: `mat2str`. Empty arrays are always shown as `[]`, `''` or `{}`. Most input characters below 32 are replaced by space. Cell arrays are converted to string representation too. Optional second, third and fourth inputs respectively specify column separator, row separator and whether the separators should be used for non-cell arrays too. Second and third inputs are converted to char if needed. *See also* `XD`, `YD`, `ZD`
`XD`	0– (‡ / 1)	0	`disp(num2str(eachInput, '%.15g '))`, where `eachInput` loops over all inputs. (Nested) cell arrays are (recursively) unboxed in linear order. Most characters below 32 are replaced by space. Symbolic data is displayed directly. *See also* `D`, `YD`, `ZD`
`YD`	1– (2 / 1)	0–2 (1 / 0)	`sprintf` with format string as last input. If 0 outputs: prints to screen using `fprintf(...)`. *See also* `D`, `XD`, `ZD`
`ZD`	0– (1)	0	`disp` for each input. For char input, most characters below 32 are replaced by space. *See also* `D`, `XD`, `YD`
`E`	1	1	`(...)*2`

XE	3	1	With numeric or char inputs, replace in first input all occurrences of each element of the second input by the corresponding element of the third input. The third input may be longer than the second, and then the extra elements are ignored. Or it may have a single element, and then it is implicitly replicated. Output has the same class and size as the first input. If the three inputs are cell arrays of strings (the second may also be a string instead of a cell array of strings), each string of the first input is considered atomic, that is, replacing is based on whole strings. If the first input is a cell array and the others are numeric or char, replacing is done on each cell's contents as if the cell's contents were the first input
ZE	1–4 (1 / 2)	1	(i) With 1, 2 or 3 inputs: ifftn. If first input is a vector and second input is a scalar, a 1 is prepended or appended to the second input according to the vector orientation (so the result is the same as that of ifft). (ii) With 4 inputs: ifft
YF	1	1–2 (1 / 2)	(i) With 1 output: exponents of prime factor decomposition, without skipping primes. If the input is a numeric array it is linearized, and each result is in an output row. If any entry in the input is negative, the absolute value is taken, and the output contains only non-zero exponents. (ii) With 2 outputs: first output gives the prime factors, second gives the exponents. Primes that are not factors are skipped, as are their zero exponents. If any entry in the input is negative, the absolute value is taken, and all intermediate primes are included, possibly with zero exponents. *See also* Yf
ZF	1–3 (1 / 2)	1	(i) With 1 or 2 inputs: fftn. If first input is a vector and second input is a scalar, a 1 is prepended or appended to the second input according to the vector orientation (so the result is the same as that of fft). (ii) With 3 inputs: fft
G	0–1 (␣ / 0)	0– (⊓)	paste from user-input clipboard G. If 0 input arguments: addresses all levels. If 1 input argument: addresses specified level. In either of those cases, if clipboard G has no levels one user-input is implicitly taken to fill the first level
XG	1– (1 / 2)	0	plot. Calls drawnow to update figure immediately. With one input, or with several inputs the second of which is a string: if the first input is complex (even with zero imaginary part), axis equal is also called.

YG	2– (2 / 3)	0	imwrite, imagesc, image or imshow. (i) If last input is a scalar: 0 corresponds to imwrite, 1 to imagesc, 2 to image and 3 to imshow. The corresponding function is called with the remaining inputs. (ii) If last input is numeric or logical and not a scalar: imshow is called with all inputs. (iii) If last input is char: imwrite is called with all inputs. (i, iii) For imwrite, the first input of type char is interpreted as file name. If it has no extension '.png' is added; if it's empty it is replaced by 'image.png'; and if non existent 'image.png' is used as final input. (i, ii, iii) For imshow and imwrite, if the second input is logical it is converted to double. If it is numeric, has the shape of a colormap, and has some entry greater than 1, it is normalized by converting to uint8, then to double, and then dividing by 255. For imagesc and image, the function call is followed by axis ij, axis image. For imagesc, image and imshow, drawnow is called to update figure immediately
ZG	1– (2 / 3)	0–1 (0)	Depending on numeric last input, calls a graphic function or format with the remaining inputs. 0: format. 1: axis. Calls drawnow to update figure immediately. Flag strings in first to second-last inputs can be replaced by numbers, as follows: 1: 'equal', 2: 'image', 3: 'square', 4: 'ij', 5: 'xy', 6: 'normal', 7: 'off', 8: 'on', 9: 'tight', 10: 'manual', 11: 'fill', 12: 'auto', 13: 'vis3d'. 2: colormap. If the first input is numeric, has the shape of a colormap, and has some entry greater than 1, it is normalized by converting to uint8, then to double, and then dividing by 255. With 3 inputs (including the last input 2), the second input specifies colormap size. With 0 outputs, calls drawnow to update figure immediately. 3: hold. Flag strings in first input can be replaced by numbers, as follows: 1: 'on', 2: 'off'. 4: grid. Flag strings in first input can be replaced by numbers, as follows: 1: 'on', 2: 'off'. 5: colorbar. If the first input is logical (not necessarily scalar) it activates or deactivates the colorbar
H	0	0– (†)	paste from clipboard H
XH	0– (1 / 2)	0	copy to clipboard H
YH	1– (2 / 4)	0	Depending on numeric last input, calls a plotting function with the remaining inputs. 0: plot3. 1: surf. 2: mesh. 3: stem. 4: stairs. 5: bar. 6: loglog. After that, drawnow is called to update figure immediately
I	0	0– (†)	paste from clipboard I
XI	0– (1)	0	copy to clipboard I

`YI`	3–4 (3 / 4)	1	`col2im`. Uses `'distinct'` option by default. Second and third inputs may be scalars, and then they are interpreted as numbers of columns. Third input may be a two-vector with product less then the number of elements of first input, and then it is appropriately scaled. This function allows flag strings in fourth input to be replaced by numbers, as follows: 1: `'distinct'`, 2: `'sliding'`
`ZI`	1– (2 / 3)	1– (1)	Depending on numeric last input, calls an image processing function with the remaining inputs. 0: `imfill`. If first input is logical or numerical it is converted to char. 1: `bwlabeln`. 2: `imdilate`. This function allows second input to be number 4, 5, 8 or 9, which is interpreted as the corresponding neighbourhood mask. 3: `imerode`. This function allows second input to be number 4, 5, 8 or 9, which is interpreted as the corresponding neighbourhood mask. 4: `bweuler`. 5: `bwselect` with 4 inputs and 1 output.
`J`	0	0– (†)	paste from clipboard J
`XJ`	0– (1)	0	copy to clipboard J
`ZJ`	1– (2 / 3)	0– (1)	Depending on numeric last input, calls a symbolic-specific function with the remaining inputs. 0: `simplify`. 1: `pretty`. 2: `latex`. 3: `numden`. *See also* `X$`
`K`	0	0– (†)	paste from clipboard K
`XK`	0– (1)	0	copy to clipboard K
`L`	1	0– (†)	paste from multi-level clipboard L. Input specifies level
`XL`	1– (2 / 3)	0	copy to multi-level clipboard L. Last input specifies level
`YL`	1– (2 / 3)	1– (1)	`gallery` with matrix name as last input. Also includes `magic`, `hilb`, `invhilb`, `hadamard`, `pascal`, `spiral`. This function allows some strings in last input to be replaced by numbers, as follows: 1: `'spiral'`, 2: `'pascal'`, 3: `'magic'`, 4: `'hadamard'`, 5: `'circul'`, 6: `'gcdmat'`, 7: `'minij'`, 8: `'hilb'`, 9: `'invhilb'`, 10: `'tridiag'`, 11: `'ris'`
`M`	1	0– (*)	paste from function-input clipboard M. Input specifies level (1 to 4) or individual input (5 or larger)
`XM`	1–2 (1)	1–3 (1 / 2nd)	`mode`. First input can be a cell array of strings
`N`	0	1	number of elements in the stack
`XN`	2	1	`nchoosek`. This interprets first input as an array (even if it is a single number). For inputs `x` and `k`, if `x` has less than `k` elements or if `k` is non-positive the result is an empty array. *See also* `Xn`
`YN`	0– (0)	1	`NaN` function. If 0 inputs: produces literal `NaN`.

ZN	1	1	isnan
O	0– (0 / 2)	1	zeros (if 0 inputs: produces output 0)
XO	1–4 (2)	1	datestr. *See also* YO, ZO
YO	1–6 (1 / 2)	1	datenum. With 2 inputs, if the second input is numeric it is interpreted as a format specifier as in datestr. *See also* XO, ZO
ZO	1–3 (1 / 2)	1–6 (1)	datevec. *See also* XO, YO
P	1–2 (1 / 2)	1	flip. *See also* XP
XP	1	1	flipud. *See also* P
YP	0	1	pi
ZP	1–5 (2 / 3)	1–2 (1)	(i) pdist2. Only predefined distance functions are allowed. This function allows flag strings in the third input to be replaced by numbers, as follows: 1: 'cityblock', 2: 'hamming', 3: 'chebychev', 4: 'correlation', 5: 'cosine', 6: 'seuclidean', 7: 'minkowski', 8: 'mahalanobis', 9: 'spearman', 10: 'jaccard', 11: 'euclidean'. (ii) With 1 input and 1 output: row vector containing all entries below the main diagonal of a matrix
Q	1	1	(...)+1
XQ	2–6 (3)	1	accumarray. If first input is char it is converted to double. The third input may be omitted, and then it is interpreted as []. Fourth/third argument specifies an anonymous function, as follows: 1: '@sum', 2: '@mean', 3: '@(x){sort(x).'}', 4: '@max', 5: '@min', 6: '@prod', 7: '@(x){x.'}', 8: '@(x){x}', 9: '@(x){sort(x)}', 10: '@(x)x(1)', 11: '@(x)x(end)', 12: '@(x){cumsum(x).'}', 13: '@(x){cumprod(x)}', 14: '@nansum', 15: '@nanmean', 16: '@nanmax', 17: '@nanmin', 18: '@(x){cummax(x).'}', 19: '@(x){cummin(x)}', 20: '@(x){cummax(x).'}', 21: '@(x){cummin(x)}'
YQ	1–2 (2)	1–2 (2)	rat
ZQ	1–4 (2 / 1)	1–2 (1)	(i) If 1 input: roots. (ii) If 2 inputs: polyval. (iii) If 3 inputs: polyfit. (iv) If 4 inputs: inpolygon
R	1–2 (1 / 2)	1	triu. *See also* XR.

XR	1–2 (1 / 2)	1	triu(..., 1). If 2 inputs: builds an upper triangular or symmetric matrix from a vector (or from an array in linear order). Second input indicates if the diagonal is filled/not and if the matrix is made symmetric, as follows. 0: don't use diagonal, don't make symmetric. 1: use diagonal, don't make symmetric. 2: don't use diagonal, make symmetric. 3: use diagonal, make symmetric. *See also* R, ZR
YR	1–2 (1 / 2)	1	tril. *See also* ZR.
ZR	1–2 (1 / 2)	1	tril(..., -1). If 2 inputs: builds a lower triangular or symmetric matrix from a vector (or from an array in linear order). Second input indicates if the diagonal is filled/not and if the matrix is made symmetric, as follows. 0: don't use diagonal, don't make symmetric. 1: use diagonal, don't make symmetric. 2: don't use diagonal, make symmetric. 3: use diagonal, make symmetric. *See also* XR, YR.
S	1–3 (1)	1–2 (1 / 2nd)	sort an array (sort) / sort an array based on another. (i) Single-array mode works like Matlab's sort. If 2 inputs, a negative value of the second input corresponds to descending order. If first input is a cell array and the first cell contains a char array, the rest of the cells' contents are converted to char. (ii) If the first input is a cell array and the first cell contains a numeric array, single-array numeric mode is used. The first input is linearized if it's not a vector, and its contents are linearized for the purposes of sorting. The first input is then sorted in lexicographic order, ignoring other inputs. (iii) In two-array mode, this function takes as first 2 inputs an array and a vector array which is not char, both with the same number of elements. If the first array is not a vector it is linearized. The second vector is sorted and its order is applied to the first. An optional third input specifies direction as a string, or as a negative number in the non-singleton dimension of the second vector. The outputs are the two sorted arrays. (In two-array mode, if the two input arrays are scalar and the second can be interpreted as a dimension there is no inconsistency with single array mode, as the result is the same). *See also* XS
XS	1–2 (1)	1–2 (1 / 2nd)	sortrows. *See also* S

YS	2–3 (2 / 3)	1	circshift. If second input is a scalar and there's no third input, the shift is applied along the first non-singleton dimension. This function also allows first input a 2D array; third input a scalar specifying dimension; and second input a vector or array specifying the shift for each position in the other dimension
ZS	1–3 (1 / 3)	1	(i) With 1 input: sign. (ii) With 2 inputs: fftshift. Second input equal to 0 corresponds to fftshift with 1 input. (iii) With 3 inputs: linspace
YT	1–2 (1 / 2)	1	toeplitz. Output is char if any input is
U	1	1–2 (1)	(i) For char input: str2num with content checking. Most characters below 32 are replaced by space (as in D). The input content is then checked. If it fails, [] is returned. Else str2num is applied. If that fails, the input string is evaluated. If that also fails, [] is returned. The second output is supported in all cases. (ii) For numeric input: (...).^2
XU	1	1	str2double
V	1–2 (1 / 2)	1	num2str. Uses format '%.15g ' by default. To get Matlab's default format use [] as format specification
W	1	1	2.^(...). Array power (or rather exponentiation) with base 2
XX	2–9 (2 / 3)	1–6 (◇)	regexp. With 2 inputs and 1 output: regexp(..., ..., 'match'). If first or second inputs are numeric they are converted to char. If they are numeric or char arrays they are linearized into a row. If they are cells their contents are converted to char. 'names' output is not supported. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: 'start', 2: 'split', 3: 'end', 4: 'once', 5: 'tokenExtents', 6: 'tokens', 7: 'match'
YX	3–5 (3 / 4)	1	regexprep. If first, second or third inputs are numeric they are converted to char. If they are numeric or char arrays they are linearized into a row. If they are cells their contents are converted to char
YY	0– (0)	1	inf function. If 0 inputs: produces literal inf.
ZY	1	1	isinf
X[	2	1– (△)	ind2sub without input restrictions

`\`	2	1–2 (1 / 2)	`mod`, element-wise with singleton expansion. With 2 outputs: second output is `floor(.../...)`. For 1 or 2 outputs, if the first input is complex the real and imaginary parts are dealt with separately. *See also* `X\`, `o`
`X\`	2	1	`mod(...-1, ...)+1`, element-wise with singleton expansion. *See also* `\`
`Y\`	2	1	left matrix division, `\` (`mldivide`)
`Z\`	1	1	divisors of a number. For negative input the absolute value is used
`X]`	3– (3)	1	`sub2ind` without input restrictions
`^`	2	1	`.^` (`power`), element-wise with singleton expansion
`X^`	1	1	`sqrt`
`Y^`	2	1	`^` (`mpower`). If the second input is an array, the output is the sum of matrix powers with those exponents
`Z^`	2	1	(i) Given an array and a number *n*, computes the Cartesian power of the array times itself *n* times. (ii) If the first input is a number and the second input is an array of at least two elements, the inputs are interpreted in reverse order. When using this mode, caution is needed in case the second input (array) may become a scalar, because then mode (i) will be used. *See also* `Z*`
`_`	1	1	(i) If input is not `uint8`: unary `-` (`uminus`), that is, output is the negative of the input. (ii) If input is `uint8`: converts to `double` and divides by 255
`Y‘`	0	0–1 (0)	`tic`. *See also* `Z‘`
`Z‘`	0–1 (0)	0–1 (1)	`toc`. *See also* `Y‘`
`a`	1–2 (1 / 2)	1	`any`. *See also* `Xa`
`Xa`	1	1	`any(..., 1)`. *See also* `a`

`Ya`	2–4 (2 / 3)	1	(i) `padarray`. It allows the first input to be `char`; and then the output is also `char`. If the second input is `logical` or the pad value is `char` they are converted to `double`. This function allows flag strings in fourth input to be replaced by numbers, as follows: 1: `'pre'`, 2: `'post'`, 3: `'both'`. (ii) If the second input contains at least one negative or complex value, the array in the first input is unpadded. The array can only have two dimensions. The second input can have one or two entries, specifying dimensions. A negative value indicates unpad along that dimension. A complex value indicates unpad along the two dimensions. The third input specifies which values are considered padding; by default 0
`Za`	3–4 (3 / 4)	1	Converts the number represented by input 1 from the base specified by input 2 to that of input 3. Each base can be a number or a vector. In the first case the alphabet is from 0 to that number minus 1. If the second or third input equals `T` or `F`, it is respectively interpreted as `' ':'~'` (all printable ASCII chars) or `[' ':'&' '(':'~']` (all printable ASCII chars except single quote). Non-valid digits in the first input are discarded. An optional fourth input indicates number of digits of the result. First input can be a matrix or a cell array; and then the result is a matrix in which each row corresponds to a row of the input matrix, or to a cell of the input cell array in linear order. *See also* `YA`, `ZA`
`b`	0– (3 / 4)	0	bubble up element in stack. *See also* `w`
`Yb`	1– (1 / 2)	1–2 (1)	(i) `strsplit`. If second input is numeric it is converted to char. This function allows flag strings in third and successive inputs to be replaced by numbers, as follows: 1: `'CollapseDelimiters'`, 2: `'DelimiterType'`, 3: `'RegularExpression'`, 4: `'Simple'`. (ii) First input can be a cell array of strings, and then the result is a cell array of cell arrays of strings. Only one output is supported in this mode. (iii) First input can be numeric or logical. In this mode only one output is supported, the default delimiter is 0 or `false`, and the parameter `'CollapseDelimiters'` can be used. The second input (delimiter) can be a single number or a numeric array, and in the latter case each element is a possible delimiter. If the first input is a row array the output will contain row arrays; otherwise it will contain column arrays
`c`	1– (1)	1	`char`. Sparse input is first converted to full, and logical input is first converted to double. For cell array input, non-char cell contents are first converted to char. In all cases, non-integer values are rounded down

Xc	3– (3 / ‡)	1	cat. The dimension is the last input
Yc	1– (2 / ‡)	1	strcat. If not all inputs are numerical or logical, numerical or logical inputs are converted to char. If all inputs are numerical or logical the result is numerical or logical (with singleton expansion along the first dimension), and no trailing values are removed
Zc	1–2 (1 / 2)	1	If cell input: strjoin. This function also allows input with numeric content. The first cell of the first input determines char or numeric mode. If that cell contains a char (resp. numeric) array, numeric (resp. char) contents in other cells, as well as the second input or its contents, are converted to char (resp. double). (i) Char mode corresponds to strjoin. (ii) Numeric mode is similar: it has a default delimiter, which is 0; or a single delimiter may be specified, which may be a scalar or an array; or a cell array of delimiters may be used. (i, ii) Both in char and in numeric mode, non-vector arrays are linearized, and the result is a row vector. Surplus delimiters are ignored. (iii) With a single, non-cell input: nonzeros are replaced by 35 (code point of '#') and the result is converted to char
d	1–3 (1 / 2)	1	diff with second and third input interchanged: second specifies dimension, third specifies difference order.
Xd	1–4 (1 / 2)	1–2 (1)	If 1 input and 1 output: diag. Otherwise: spdiags, with char inputs automatically converted to double. With 2 inputs and 1 output, if the second input is true it selects all diagonals from the first input
Yd	1– (2 / ‡)	1	blkdiag
Zd	1–2 (2 / 1)	1–3 (1)	gcd, element-wise with singleton expansion. With 1 input and 1 output, computes the greatest common divisor of all elements of the input

`e`	1– (2 / 1)	1	(i) With 1 input: `squeeze`. (ii) With more than 1 input: `reshape`. If second input is logical, inputs beyond the second are ignored. Contiguous equal values in the second input indicate dimensions that will be collapsed; and a final `[]` is implicit. If there are 2 inputs and the second is non-logical and empty, it is replaced by the rounded-up square root of the number of elements of the first input, repeated; thus a square matrix is returned. If there are 2 inputs and the second is a non-logical scalar, a final `[]` is implicit. If size specification doesn't contain `[]` (explicit or implicit), the first input is padded or truncated if needed; padding is done with `0`, `false`, `char(0)` or `{[]}` according to type of first input. If size specification contains `[]` (explicit or implicit), the first input is padded if needed
`Ye`	1–2 (1 / 2)	1	(i) With one numerical or symbolic input: `expm`. (ii) With one logical input `M`: the input is interpreted as the adjacency matrix of a graph, which is made square if needed. The `logical` version of the adjacency matrix of the power of the graph is computed with increasing exponent until the result no longer changes. This is the same as `logical(M*expm(M))` except that numerical precision issues are avoided. (iii) With a logical input `M` and a non-negative integer input `n`: `M` interpreted as the adjacency matrix of a graph, which is made square if needed. The `logical` version of the adjacency matrix of the power of the graph is computed with exponent `n`. *See also* `Y^`
`Ze`	1–2 (1 / 2)	1	(i) For 1 input: `exp`. (ii) For 2 inputs: Levenshtein distance between char, logical or numeric inputs. Either input can be a cell array, and then the result is a vector or matrix of distances.
`f`	1–3 (1)	1–3 (1 / 2)	`find`
`Xf`	1–4 (2 / 1)	1	(i) If 2 or more inputs: `strfind`. Works also when the first input is a numeric array or a cell array of numeric arrays. In this case each numeric array is linearized, and results are row vectors. (ii) If 1 input: output is a cell array of all non-empty substrings. Works also for numeric or cell arrays
`Yf`	1	1	(i) For a scalar numeric or symbolic input: `factor`. Input 1 gives an empty array, not 1 like `factor` does. For negative input, `factor` is applied to the absolute value of the input (so input -1 gives 1). (ii) For a numeric or symbolic input with two elements: divides each element by the greatest common divisor. *See also* `YF`

`g`	1	1	(i) For non-cell input: `logical`. Works also for complex numbers, using their absolute value. (ii) For cell input: `cell2mat`
`Xg`	1– (2)	1– (□)	`ndgrid`
`Yg`	1–3 (1)	1	`gamma` / `gammainc` / `betainc`, depending on number of inputs
`Zg`	1–2 (1 / 2)	1	`gammaln` / `betaln`, depending on number of inputs
`h`	0– (2 / ‡)	1	`horzcat`. If inputs are matrices with non-matching sizes they are linearized
`Xh`	0– (‡ / 2)	1	concatenate into cell array (`{..., ...}`)
`Yh`	1–2 (2 / 1)	1	`hankel`. If any input is of type char: returns char output
`Zh`	3	1	`hypergeom`. If a numerator coefficient is a nonpositive integer the series is truncated at the first index for which a zero appears in the numerator, regardless of the denominator
`i`	0–2 (0)	1	`input` with content checking. If 0 inputs: uses default prompt string. *See also* `j`.
`Xi`	1–5 (1)	1–2 (1)	`urlread`. If the input string doesn't contain `'://'`, the string `'http://'` is prepended
`Yi`	1– (1)	1–3 (1 / 2)	`imread`
`j`	0–1 (0)	1	`input(..., 's')`. If 0 inputs: uses default prompt string. *See also* `i`
`Xj`	1	1	`real`
`Yj`	1	1	`imag`
`Zj`	1	1–2 (1 / 2)	(i) With one output: complex conjugate (`conj`). (ii) With two outputs: real and imaginary parts (`real`, `imag`)
`k`	1	1	`lower` for strings or cell arrays; `floor` for numerical or symbolic arrays. *See also* `Xk`
`Xk`	1	1	`upper` for strings or cell arrays of strings; `ceil` for numerical or symbolic arrays. *See also* `k`
`Yk`	2	1–3 (1 / 3rd)	closest value in second input to each value in first input, in terms of absolute difference. For equidistant values it picks the one that appears first (in linear order) in the second input. Second output gives resulting absolute differences. Third output gives linear indices of selected elements
`l`	0– (0 / 2)	1	`ones` (if 0 inputs: produces output 1)

Xl	2–3 (2 / 3)	1	(i) With 2 inputs, the minimum of the two inputs is computed, element-wise with singleton expansion. (ii) With 3 inputs, the function computes the minimum between the first and the second, and then the maximum between that and the third, element-wise with singleton expansion. (i, ii) Output is char if first input is. *See also* X<, X>
Yl	1–2 (1 / 2)	1	log. If 2 inputs: second input specifies logarithm base
Zl	1	1–2 (1)	log2
m	2–4 (2)	1–2 (1 / 2nd)	ismember. It also works for cell arrays with numeric content, or with mixed char/numeric content. *See also* Xm
Xm	2–3 (2)	1–2 (1 / 2nd)	ismember(..., 'rows', ...). *See also* m
Ym	1–4 (1 / 2)	1	mean. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: 'omitnan', 2: 'includenan', 3: 'double', 4: 'native', 5: 'default'
Zm	1–2 (2 / 1)	1	lcm, element-wise with singleton expansion. With 1 input, computes the least common multiple of all elements of the input
n	1	1– (1 / ×)	numel. For several outputs, or a single output other than the first, gives size along those dimensions
Xn	1–2 (2 / 1)	1	(i) If 2 inputs: nchoosek. This interprets first input as number(s). If the inputs are arrays, the function is computed element-wise with singleton expansion. For values n and k in first and second inputs, if n is less than k the result is 0. (ii) If 1 input: multinomial coefficient. The input contains the lower terms; their sum is the upper term. *See also* XN
Yn	1–5 (2 / 4)	1	(i) If 1 input: poly. (ii) If 2 or more inputs: interp1. With 2 or 3 numeric inputs, flags 'linear' and 'extrap' are assumed. 'pp' option not supported. A finite numeric value in the fourth input is replaced by a flag string, as follows: 1: 'linear', 2: 'nearest', 3: 'next', 4: 'previous', 5: 'spline', 6: 'pchip', 7: 'cubic', 8: 'v5cubic'
o	1–3 (1 / 2)	1	(i) With one non-cell array, non-double input: double. (ii) For cell array input behaves similarly to char: linearizes cell array, converts cell contents to double and concatenates vertically, padding if needed. By default, padding is with zeros on the right. Optional second input indicates fill side: left if it evaluates to false, right if it evaluates to true. Third optional input specifies fill value. (iii) With one input of type double: mod(..., 2)

`Xo`	1	1	`int64`
`Yo`	1–3 (1 / 2)	1	(i) For numeric arrays: `round`. Third input can be a number instead of a string, as follows: 1: `'significant'`, 2: `'decimals'`. (ii) For character arrays or cell arrays of strings: change case. Optional second input specifies in which characters or cells the case will be changed
`Zo`	1	1	`fix`. *See also* `k`, `Xk`
`p`	1–3 (1 / 2)	1	`prod`. If first input is `char` it is converted to `double`. *See also* `Xp`
`Xp`	1–3 (1)	1	`prod(..., 1, ...)`. If first input is `char` it is converted to `double`. *See also* `p`.
`Yp`	1–4 (1 / 2)	1	`cumprod`. Allows first input to be char. If 3 numeric inputs: the third input is interpreted as a minimum which is applied after the product at each position. If 4 numeric inputs: last two inputs are minimum and maximum respectively
`Zp`	1	1	For input with non-negative entries: `isprime`. For input with non-positive entries: Euler's totient function for absolute value of each entry
`q`	1	1	`(...)-1`
`Xq`	2–3 (2 / 3)	1	`quantile`
`Yq`	1	1	For input with positive entries: *n*-th prime for each value *n* in the input array. For input with non-positive entries: next prime for absolute value of each entry. *See also* `Zq`
`Zq`	1	1	`primes`. *See also* `Yq`
`r`	0– (0 / 2)	0–1 (1)	`rand`
`Xr`	0– (0 / 2)	0–1 (1)	`randn`
`Yr`	1– (1 / 2)	1	`randi`. If the first input is a vector of 2 elements it is automatically sorted. With 2 inputs, if the second input is a scalar `n` it is transformed into `[1 n]`
`Zr`	1–4 (2 / 3)	1	(i) With 2 or more inputs: `randsample`. Non-vector first input is linearized to a column. Does not support stream specification. (ii) With 1 input: shuffle the input array, that is, apply a random permutation to it. This is done linearly, that is, regardless of dimensions. The shape of the array is preserved.

`s`	1–4 (1 / 2)	1	`sum`. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'omitnan'`, 2: `'includenan'`, 3: `'double'`, 4: `'native'`, 5: `'default'`. *See also* `Xs`
`Xs`	1–3 (1)	1	`sum(..., 1, ...)`. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'omitnan'`, 2: `'includenan'`, 3: `'double'`, 4: `'native'`, 5: `'default'`. *See also* `s`
`Ys`	1–4 (1 / 2)	1	`cumsum`. Allows first input to be char. Optional third and fourth numerical scalar inputs are interpreted as minimum and maximum values, which are applied after the sum at each position. These can be substituted by a 2-element numerical array as second or third input
`Zs`	1–4 (1 / 2)	1	(i) `std`. (ii) With 2 inputs `x`, `k` and `k` equal to 2 or 3, gives `cov(x, k-2)`. (iii) With 2 inputs `x`, `k` and `k` equal to 4 or 5, gives `skewness(x(:), k-4)`. (iv) With 2 inputs `x`, `k` and `k` equal to 6 or 7, gives `kurtosis(x(:), k-6)`
`t`	0– (1)	0	duplicate elements in stack. The duplicated elements are those specified as inputs. *See also* `y`
`Zt`	3	1	(i) If first input is char or a cell array: `strrep`. Numerical second and third inputs are converted to char. If the first input is char it is linearized into a row vector. (ii) The first input can be a numerical or logical array (not a cell array of numeric or logical arrays). In this case the other inputs can be char, numeric or logical (not cell); each input array is linearized; result is a row vector; and result has the type of the third input, even if no substitutions have been actually done
`u`	1–4 (1)	1–4 (1 / 1st and 4th)	`unique`, with `'stable'` option by default. It includes a fourth output with the count for each unique element. It also allows cell array input with numeric content, or mixed char/numeric content. In this case it only works with a single output, either in sorted or stable mode. This function allows flag strings in second and subsequent inputs to be replaced by numbers, as follows: 1: `'stable'`, 2: `'sorted'`, 3: `'last'`, 4: `'first'`, 5: `'rows'`. *See also* `Xu`
`Xu`	1–3 (1)	1–4 (1)	`unique(..., 'rows', ...)`, with `'stable'` option by default. It includes a fourth output with the count for each unique row. This function allows flag strings in second and subsequent inputs to be replaced by numbers, as follows: 1: `'stable'`, 2: `'sorted'`, 3: `'last'`, 4: `'first'`. *See also* `u`

`Zu`	2	1	`strjust`. Strings in second input can be replaced by numbers, as follows: 1: `'right'`, 2: `'left'`, 3: `'center'`
`v`	0– (‡ / 2)	1	`vertcat`. If inputs are matrices with non-matching sizes they are linearized
`Yv`	1–2 (1)	1–3 (1 / 2nd)	(i) If input is numeric or symbolic: with 1 or 2 outputs: `eig`. With 3 outputs: `svd`. If output specification is `FTF` (keep second output only), `svd` is called with a single output, which gives a vector with the singular values. (ii) If input is a string or cell array of strings: `strtrim`. For string or char array input, char 0 also counts as whitespace
`Zv`	1–2 (1 / 2)	1	(i) For one scalar numeric or symbolic input `n`: symmetric range. For nonnegative `n` it rounds down and then gives `[1:n n-1:-1:1]`; for negative `n` it removes sign, rounds down and gives `[n:-1:1 2:n]`. (ii) For two numeric inputs, or a logical and a numeric input: applies symmetric range indexing to the first input along each dimension specified by the (linearized) second input. The indexing along a given dimension doesn't repeat the last entry if the number specifying the dimension is positive, and repeats it if negative. (iii) For one string or cell array input: `deblank`
`w`	0– (2)	0	swap elements in stack. *See also* `b`
`x`	0– (1)	0	delete elements from stack. The deleted elements are those specified as inputs
`Xx`	0–1 (0 / 1)	0	`clc`. With 1 input `x`: `pause(x/10)` and then `clc`. Inputs `F` and `T` are interpreted as `2.5` and `10` respectively
`Zx`	0	0	resets stack, input/output specifications and clipboards to their initial values
`y`	0– (2 / 3)	0	duplicate one element in stack. The duplicated element is the lowest among those specified as inputs, if any. *See also* `t`
`Xy`	1–4 (1)	1	`eye` (matrix with ones on diagonal and zeros elsewhere)
`Yy`	2	1	`hypot`, element-wise with singleton expansion
`Zy`	1–2 (1)	1– (1)	`size`
`z`	1	1	(i) For non-cell input: `nnz`: number of nonzero elements. (ii) For cell array input: `cellfun(@nnz, ...)`: number of nonzero elements of each cell
`Xz`	1	1	`nonzeros`. If input is a string or char array: removes whitespace, including `char(0)`; the result is a string. If input is cell array of strings: operates on each string
`X{`	1–2 (1 / 2)	1	`num2cell`

`Y{`	2– (3 / 2)	1	`mat2cell`
`Z{`	1	1	`cellstr` for `char` input, or `mat2cell(x, ones(size(x,1),1), size(x,2),...,size(x,ndims(x)))` for non-`char` arrays of any number of dimensions
`|`	1–2 (1 / 2)	1	If 1 input: `abs`. If 2 inputs, second input non-zero: `norm`. If 2 inputs, second input zero: `det`, or `sqrt(det(A*A.'))` for first input `A` non-square
`X|`	2–4 (2 / 3)	1–3 (1)	`union`. Uses the `'stable'` flag by default. If one input is char and the other is numeric, the latter is converted to char. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'rows'`, 2: `'stable'`, 3: `'sorted'`
`Y|`	0– (2 / ‡)	1	`|` (`or`), element-wise with singleton expansion
`Z|`	2–3 (2)	1	`bitor` (bitwise 'or'), element-wise with singleton expansion. If first or second inputs are `char` they are converted to `double`. `double` inputs are rounded. If the inputs are `double` they can have negative values, and in that case they must be in the range from `-2^52` up to `2^52-1`. *See also* `Z&`, `Z~`
`Z}`	1–2 (1 / 2)	0– (▽)	split array into subarrays along the first non-singleton dimension. With 2 inputs: split into subarrays along the dimension indicated by the second input
`~`	1	1	`~` (`not`)
`X~`	2–4 (2 / 3)	1–3 (1)	`setxor`. Uses the `'stable'` flag by default. If one input is char and the other is numeric, the latter is converted to char. This function allows flag strings in third and subsequent inputs to be replaced by numbers, as follows: 1: `'rows'`, 2: `'stable'`, 3: `'sorted'`. *See also* `X-`
`Y~`	2	1	`xor`, element-wise with singleton expansion
`Z~`	2–3 (2)	1	`bitxor` (bitwise 'xor'), element-wise with singleton expansion. If first or second inputs are `char` they are converted to `double`. `double` inputs are rounded. If the inputs are `double` they can have negative values, and in that case they must be in the range from `-2^52` up to `2^52-1`. *See also* `Z&`, `Z|`

† Number of elements in clipboard (H, I, J, K), or in clipboard level (L).

‡ Number of elements in stack.

* Number of elements in clipboard level for combined addressing; or 1 for individual addressing.

△ Number of elements of first input.

▽ Number of elements or arrays that will be produced.

⌴ 0 if clipboard currently has 0 or 1 levels; 1 otherwise.

⌐ Number of levels addressed according to input specification.

◊ According to specified keywords.

□ Number of inputs.

× Number of dimensions of first input.

Functions `X0`...`Z9` produce predefined literals depending on their numeric input. These are specified in Table 7.

Table 7: Output of predefined literal functions

Function X0					
1	'stable'	2	'sorted'	3	'rows'
4	'last'	5	'reverse'	6	'descend'
10	'first'	11	'forward'	12	'legacy'
13	'ascend'				

Function X1					
1	'bank'	2	'compact'	3	'hex'
4	'short'	5	'shortg'	6	'long'
7	'longg'	8	'loose'	9	'rat'
10	'longe'	11	'longeng'	12	'shorte'
13	'shorteng'				

Function X2					
1	'double'	2	'int8'	3	'int64'
4	'uint8'	5	'uint64'	6	'char'
7	'logical'	8	'single'	10	'int16'
11	'int32'	12	'uint16'	13	'uint32'
14	'like'	15	'decimals'	16	'includenan'
17	'native'	18	'omitnan'	19	'significant'

Function X3					
1	'start'	2	'end'	3	'tokenExtents'
4	'match'	5	'tokens'	6	'names'
7	'split'	8	'once'	10	'ignorecase'
11	'preservecase'				

Function X4					
1	'CollapseDelim iters'	2	'DelimiterType'	3	'RegularExpression'

Table 7: Output of predefined literal functions—*continued*

Function X5					
1	'full'	2	'same'	3	'valid'
6	'linear'	7	'nearest'	8	'next'
9	'previous'	10	'spline'	11	'pchip'
12	'cubic'	13	'v5cubic'	14	'extrap'
20	'distinct'	21	'sliding'	22	'indexed'

Function X6					
1	'get'	2	'post'	3	'Timeout'
4	'png'	5	'ppm'	6	'gif'
7	'bmp'	9	'none'	11	'Frames'
12	'BackgroundColor'	13	'Index'	14	'Info'
15	'ReductionLevel'	16	'PixelRegion'	17	'V79Compatible'
20	'Border'	21	'Colormap'	22	'DisplayRange'
23	'InitialMagnification'	24	'Parent'	25	'Reduce'
30	'BitsPerSample'	31	'BitRate'	32	'Quality'
33	'maximal'	34	'ispacked'	35	'notpacked'

Function X8					
1	'cityblock'	2	'minkowski'	3	'chebychev'
4	'cosine'	5	'correlation'	6	'hamming'
10	'euclidean'	11	'seuclidean'	12	'mahalanobis'
13	'spearman'	14	'jaccard'	15	'fro'
16	'Smallest'	17	'Largest'		

Function X9					
1	'%.15g '	10	'seed'		

Table 7: Output of predefined literal functions—*continued*

Function Y0					
1	'Color'	2	'LineStyle'	3	'LineWidth'
4	'Marker'	5	'MarkerSize'	6	'on'
7	'off'	8	'CDataMapping'	9	'scaled'
10	'manual'	11	'tight'	12	'fill'
13	'ij'	14	'xy'	15	'equal'
16	'image'	17	'normal'	20	'auto'
21	'LineJoin'	22	'chamfer'	23	'miter'
24	'round'	25	'square'	26	'diamond'
27	'pentagram'	28	'hexagram'	29	'visible'
30	'Clipping'	31	'MarkerEdgeColor'	32	'MarkerFaceColor'
40	'direct'	41	'AlphaData'	42	'AlphaDataMapping'
43	'Cdata'	44	'Xdata'	45	'Ydata'
46	'ZData'	50	'autumn'	51	'bone'
52	'colorcube'	53	'cool'	54	'copper'
55	'flag'	56	'gray'	57	'hot'
58	'hsv'	59	'jet'	60	'lines'
61	'parula'	62	'pink'	63	'prism'
64	'spring'	65	'summer'	66	'winter'
70	'filled'	71	'grouped'	72	'stacked'

Table 7: Output of predefined literal functions—*continued*

Function Y2					
1	'A':'Z'	2	'a':'z'	3	['A':'Z' 'a':'z']
4	'0':'9'	5	['0':'9' 'A':'F']	6	' ':'~'
7	[' ':'&' '(':'~']	8	['0':'9' 'A':'Z' 'a':'z']	9	['0':'9' 'A':'Z' '_' 'a':'z']
10	['0':'9' 'A':'Z']	11	'aeiou'	12	'AEIOU'
13	'aeiouAEIOU'	14	'bcdfghjklmn pqrstvwxyz'	15	'BCDFGHJKLMN PQRSTVWXYZ'
16	'bcdfghjklmn pqrstvwxyzBCD FGHJKLMNPQRSTVWXYZ'	17	'aeiouy'	18	'AEIOUY'
19	'aeiouyAEIOUY'	20	['Mon'; 'Tue'; 'Wed'; 'Thu'; 'Fri'; 'Sat'; 'Sun']	21	[298 302 288 305 289 296 310]
22	{'January' 'Febru ary' 'March' 'April' 'May' 'June' 'July' 'Au gust' 'September' 'October' 'Novem ber' 'December'}	23	{'Monday' 'Tuesday' 'Wednesday' 'Thurs day' 'Friday' 'Sat urday' 'Sunday'}		

Table 7: Output of predefined literal functions—*continued*

Function Y3					
1	'magic'	2	'hilb'	3	'invhilb'
4	'hadamard'	5	'pascal'	6	'spiral'
10	'binomial'	11	'cauchy'	12	'chebspec'
13	'chebvand'	14	'chow'	15	'circul'
16	'clement'	17	'compar'	18	'condex'
19	'cycol'	20	'dorr'	21	'dramadah'
22	'fiedler'	23	'forsythe'	24	'frank'
25	'gcdmat'	26	'gearmat'	27	'grcar'
28	'hanowa'	29	'house'	30	'integerdata'
31	'invhess'	32	'invol'	33	'ipjfact'
34	'jordbloc'	35	'kahan'	36	'kms'
37	'krylov'	38	'lauchli'	39	'lehmer'
40	'leslie'	41	'lesp'	42	'lotkin'
43	'minij'	44	'moler'	45	'neumann'
46	'normaldata'	47	'orthog'	48	'parter'
49	'pei'	50	'poisson'	51	'prolate'
52	'qmult'	53	'randcolu'	54	'randcorr'
55	'randhess'	56	'randjorth'	57	'rando'
58	'randsvd'	59	'redheff'	60	'riemann'
61	'ris'	62	'sampling'	63	'smoke'
64	'toeppd'	65	'toeppen'	66	'tridiag'
67	'triw'	68	'uniformdata'	69	'wathen'
70	'wilk'				

Function Y4					
1	'\d+'	2	'[+-]?\d+(\.\d*)?'	3	'[A-Za-z]+'
4	'[A-Za-z\-]+'	5	'[A-Za-z\-\d]+'	6	'\w+'
7	'[a-z]'	8	'[A-Z]'	9	'[A-Za-z]'

Table 7: Output of predefined literal functions—*continued*

Function `Y6`					
`1`	`[false true false; true false true; false true false]`	`2`	`[false true false; true true true; false true false]`	`3`	`[true true true; true false true; true true true]`
`4`	`[true true true; true true true; true true true]`	`5`	`[true true; true true]`	`6`	`[false false false; false true false; false false false]`